

AURA: Arquitectura local-first para auditoría inteligente de calidad del dato con diagnóstico asistido por LLM

Campo	Detalle
Trabajo fin de estudio presentado por:	Joseph David Gari Bustos
Tipo de trabajo:	Desarrollo de Software
Director/a:	Luis Guadalupe Macias Trejo
Fecha:	15 de julio de 2026

Declaración de autoría y uso de herramientas de IA

Este uso se declara de forma explícita para permitir su valoración académica. No se afirma que exista una autorización previa formal del director para el empleo de IA generativa; por esa razón, la versión de predepósito debe ser revisada con el director antes del depósito definitivo. El autor asume la verificación final de cada afirmación, cita, figura, tabla y conclusión.

Resumen

La calidad del dato condiciona la fiabilidad de la analítica y de los sistemas de inteligencia artificial, pero las herramientas existentes suelen separar el perfilamiento, la explicación, la remediación y la verificación. Este Trabajo Fin de Máster presenta AURA, una arquitectura local-first para auditar archivos CSV, producir evidencia determinista y utilizar modelos de lenguaje sin cederles autoridad sobre los datos ni sobre las transformaciones. AURA ejecuta el perfilamiento en el navegador, empaqueta tres niveles de evidencia, exige una respuesta JSON validada, genera un plan de remediación mediante reglas cerradas, incorpora revisión humana y materializa un script Python/Pandas determinista. La ejecución se realiza sobre una copia y conserva bundle, recibo, hashes y reauditoría antes/después. La evaluación combina pruebas unitarias, typecheck, build, recorridos E2E y dos campañas de laboratorio sobre un dataset con ground truth congelado. La campaña definitiva comparó tres modelos cuantizados y tres métodos de entrada en 27 diagnósticos, además de nueve calentamientos excluidos. Se obtuvieron 20 corridas válidas y siete fallos conservados. Qwen3.5 4B con Contexto mínimo alcanzó el mayor índice equilibrado (92,6/100), mientras que Gemma 4 E4B destacó en calidad, estabilidad y trazabilidad, y SmoLLM3 3B fue más rápido pero menos fiable. Los resultados no establecen un ganador universal: muestran que la combinación adecuada depende del objetivo, del dataset, del hardware y del contrato. AURA aporta una cadena verificable desde el hallazgo hasta la corrección de una copia, con límites explícitos de generalización.

Palabras clave: calidad del dato; arquitectura local-first; modelos de lenguaje; trazabilidad; revisión humana.

Abstract

Data quality conditions the reliability of analytics and artificial-intelligence systems, yet existing tools often separate profiling, explanation, remediation and verification. This Master's Thesis presents AURA, a local-first architecture for auditing CSV files, producing deterministic evidence and using language models without granting them authority over data or transformations. AURA profiles the dataset in the browser, builds three controlled evidence levels, requires a validated JSON response, creates a remediation plan through a closed rule catalogue, incorporates human review and renders a deterministic Python/Pandas script. Execution occurs on a copy and preserves a bundle, receipt, hashes, and a before/after re-audit. Evaluation combines unit tests, type checking, production builds, end-to-end browser journeys and two laboratory campaigns over a frozen ground-truth dataset. The definitive campaign compared three quantized models and three input methods across 27 evaluated diagnoses, plus nine excluded warm-ups. It produced 20 valid runs and retained seven failures. Qwen3.5 4B with Minimum context achieved the highest balanced index (92.6/100); Gemma 4 E4B stood out in quality, reliability and traceability; and SmolLM3 3B was faster but less reliable. The results do not establish a universal winner: the appropriate combination depends on the objective, dataset, hardware and contract. AURA contributes a verifiable chain from findings to correction of a copy, with explicit limits on generalisation.

Keywords: data quality; local-first architecture; large language models; traceability; human review.

Índice de contenidos

1. Introducción.....	9
2. Contexto y estado del arte.....	11
2.1. Contexto del problema.....	11
2.2. Estado del arte.....	11
2.3. Comparativa de herramientas frente a AURA.....	12
2.4. Inferencia local, modelos compactos y cuantización.....	12
2.5. Riesgos de la generación directa de código mediante LLM.....	13
2.6. Evaluación reproducible de modelos de lenguaje.....	14
2.7. Brecha de investigación y posición de AURA.....	15
3. Objetivos concretos y metodología de trabajo.....	16
3.1. Objetivo general.....	16
3.2. Objetivos específicos.....	16
3.3. Metodología del trabajo.....	16
3.4. Preguntas de investigación y variables.....	17
3.5. Diseño de las campañas y congelación del protocolo.....	18
3.6. Integridad académica, reproducibilidad y uso de IA.....	18
4. Marco normativo.....	20
4.1. Protección de datos de carácter personal.....	20
4.2. Minimización y procesamiento local.....	20
4.3. Ética en IA y gobernanza.....	21
4.4. Integridad criptográfica, trazabilidad y responsabilidad humana.....	21
4.5. Integridad criptográfica y trazabilidad.....	22
5. Desarrollo específico de la contribución.....	23
5.1. Identificación de requisitos.....	23
5.1.1. Requisitos funcionales.....	23
5.1.2. Requisitos no funcionales.....	23
5.2. Diccionario funcional de componentes de AURA.....	24
5.3. Arquitectura y funcionamiento general.....	25
5.3.1. Flujo operativo actual de AURA.....	26
5.3.2. Proveedor local y adaptadores de diagnóstico.....	26
5.3.3. Arquitectura funcional final.....	27
5.3.4. Frontera de autoridad entre el LLM, AURA y la persona.....	28
5.3.5. Generación determinista del script Python/Pandas.....	28
5.3.6. Descarga, ejecución externa y dataset corregido.....	29
5.3.7. Importación verificable y reauditoría.....	30
5.4. Evidencia visual del prototipo.....	30
5.5. Fragmentos de código explicados.....	33
5.5.1. Lectura local del CSV.....	34
5.5.2. Regla determinista representativa.....	34
5.5.3. Construcción del paquete de evidencia.....	34
5.5.4. Generación de script desde acciones aprobadas.....	35
5.5.5. Revisión humana y bloqueo.....	35
5.5.6. Bloqueo de acciones no aprobadas.....	36
5.5.7. Registro canónico de columnas como autoridad.....	36
5.5.8. Plantillas deterministas y trazabilidad dentro del script.....	36
5.5.9. Bundle y recibo de ejecución con hashes.....	37
5.5.10. Reauditoría del CSV importado.....	37

5.6. Contribuciones técnicas e innovaciones de AURA.....	37
5.7. Estrategia de pruebas y aseguramiento de calidad.....	38
5.8. Decisiones de arquitectura y compromisos de diseño.....	39
5.9. Ciclo de vida del dato y fronteras de confianza.....	40
5.10. Estados, invalidación y recuperación del flujo.....	42
5.11. Estrategia de pruebas por capas.....	42
5.12. Auditoría de la evidencia exportada.....	44
6. Código fuente, datos y reproducibilidad.....	45
6.1. Código fuente.....	45
6.2. Datos analizados.....	45
6.3. Ground truth y evidencia canónica.....	46
6.4. Verificación del software y evidencia de extremo a extremo.....	46
6.5. Expediente reproducible y privacidad.....	47
6.6. Entorno congelado de la campaña definitiva.....	47
7. Diseño experimental del Laboratorio.....	49
7.1. Campaña 1: piloto de calibración.....	49
7.2. Campaña 2: protocolo definitivo.....	50
7.3. Métodos de entrada.....	51
7.4. Métricas automáticas.....	52
7.5. Validez y amenazas previstas.....	52
7.6. Selección de representantes y agregación.....	53
7.7. Amenazas a la validez experimental.....	54
8. Resultados y discusión.....	56
8.1. Panorama de la matriz.....	56
8.2. Índice equilibrado.....	57
8.3. Calidad, evidencia y claims.....	58
8.4. Velocidad y estabilidad.....	59
8.5. Configuración recomendada y transferencia al pipeline.....	60
8.6. Resultado del pipeline de remediación.....	61
8.7. Respuesta a las preguntas de investigación.....	61
8.8. Lectura de resultados por pregunta de investigación.....	62
8.9. Implicaciones prácticas y escenario de uso.....	63
8.10. Contraste con el estado del arte.....	64
9. Conclusiones.....	65
9.1. Cumplimiento del objetivo general.....	65
9.2. Cumplimiento de los objetivos específicos.....	65
9.3. Aportaciones del trabajo.....	66
9.4. Síntesis final.....	66
10. Limitaciones y trabajo futuro.....	67
10.1. Limitaciones.....	67
10.2. Trabajo futuro.....	67
10.3. Riesgos de adopción.....	68
Referencias bibliográficas.....	69
Anexo A. Parámetros y configuración reproducible.....	71
Anexo B. Glosario para lectura de resultados.....	72
Anexo C. Artefactos y rutas de evidencia.....	73
Anexo D. Frontera de claims.....	74
Anexo E. Declaración ampliada de herramientas.....	75

Índice de figuras

Figura 1. Página principal de AURA y propuesta de valor.....31

Figura 2. Arquitectura funcional y frontera de autoridad.....31

Figura 3. Ejecución de una corrida y verificación del modelo cargado en Ollama.....32

Figura 4. Matriz de modelos, entradas y repeticiones de la campaña definitiva.....32

Figura 5. Traducción de un resultado del Laboratorio a parámetros del pipeline normal.....33

Figura 6. Ejecución Python verificada y reauditoría de la copia corregida.....33

Figura 7. Evolución del piloto al protocolo definitivo.....50

Figura 8. Corridas válidas y fallidas por combinación.....56

Figura 9. Índice equilibrado por modelo y método de entrada.....57

Figura 10. Dimensiones del método Contexto mínimo.....58

Figura 11. Relación entre latencia mediana y fiabilidad.....59

Figura 12. Configuración recomendada para el siguiente diagnóstico.....60

Índice de tablas

Tabla 1. Correspondencia entre preguntas, métricas y evidencia..... 18

Tabla 2. Capas de prueba y evidencia aportada..... 39

Tabla 3. Relación entre restricciones, decisiones y consecuencias.....40

Tabla 4. Fronteras de confianza del flujo AURA.....41

Tabla 5. Capas de verificación y propósito..... 43

Tabla 6. Artefactos del expediente de Laboratorio.....48

Tabla 7. Problemas detectados en el piloto y ajustes aplicados.....50

Tabla 8. Diseño congelado de la campaña 2..... 51

Tabla 9. Definiciones operativas de las métricas..... 52

Tabla 10. Reglas de agregación de la campaña definitiva.....54

Tabla 11. Resultados por modelo y método..... 57

Tabla 12. Respuesta sintética a las preguntas de investigación..... 61

Tabla 13. Síntesis de respuestas a las preguntas de investigación.....62

Tabla 14. Trazabilidad de los seis objetivos específicos..... 65

Tabla 15. Configuración seleccionada para el pipeline normal.....71

Tabla 16. Glosario operativo..... 72

Tabla 17. Rutas de evidencia utilizadas en la memoria.....73

Tabla 18. Afirmaciones permitidas y no permitidas..... 74

1. Introducción

1.1. Motivación

1.2. Planteamiento del trabajo

AURA se plantea como una arquitectura local-first para auditoría inteligente de calidad del dato. El sistema no sustituye al analista ni delega la limpieza en un modelo de lenguaje. Distribuye responsabilidades: el motor determinista detecta hechos; el paquete de evidencia controla qué contexto puede consultar el modelo; el diagnóstico asistido interpreta sin crear acciones; el plan de remediación se construye con un vocabulario cerrado; la persona aprueba o rechaza; y un renderer determinista produce el script. Esta separación responde a un principio central: una explicación probabilística no debe convertirse automáticamente en una modificación de datos.

El producto se implementa como aplicación web y procesa el CSV en el navegador. Cuando se habilita un diagnóstico local, AURA se comunica con Ollama en el equipo del usuario y conserva el identificador solicitado y observado del modelo. La evidencia exportada permite reconstruir el método de entrada, el prompt, la respuesta cruda, la validación, el plan, el script aprobado, el bundle de ejecución, el recibo y la reauditoría. El alcance experimental del Laboratorio se limita al diagnóstico LLM: los scripts y la revisión humana pertenecen al pipeline normal y no influyen en la comparación de modelos.

La memoria estudia tanto la utilidad del flujo completo como la confiabilidad del diagnóstico estructurado. El resultado no se expresa como una promesa de limpieza autónoma, sino como una cadena verificable que distingue hechos, inferencias, decisiones y ejecución.

1.3. Alcance y preguntas de investigación

La pregunta general es: ¿cómo integrar auditoría determinista, diagnóstico asistido por modelos de lenguaje y remediación controlada sin perder privacidad, reproducibilidad ni autoridad humana? La respuesta se descompone en cuatro preguntas operativas: RQ1, si el motor identifica de forma reproducible las anomalías anotadas; RQ2, si el modelo devuelve un diagnóstico válido y anclado a la evidencia; RQ3, si una acción aprobada puede convertirse en un script y una ejecución verificables; y RQ4, qué combinación modelo-método de entrada resulta más conveniente para distintas prioridades.

El alcance incluye archivos CSV, perfilamiento local, reglas deterministas, diagnóstico estructurado con Ollama, generación controlada de scripts Python/Pandas, revisión humana, ejecución externa sobre una copia, recibos criptográficos, reauditoría y un Laboratorio de 27 diagnósticos. No incluye despliegue empresarial multiusuario, gestión centralizada de identidades, validación clínica o financiera, ni una garantía de corrección semántica para cualquier dominio. Tampoco se presenta la campaña como benchmark universal: utiliza un único dataset controlado y tres repeticiones por celda.

La unidad de análisis del Laboratorio es una corrida diagnóstico, no una acción de limpieza. Esta frontera evita que el desempeño del script, la decisión humana o la ejecución Python contaminen la evaluación del LLM. Por el contrario, en el pipeline normal la revisión humana sí es obligatoria cuando una transformación puede modificar el dataset.

1.4. Contribución y estructura de la memoria

La contribución principal es una arquitectura de autoridad limitada. El LLM no ve el CSV completo, no inventa identificadores válidos, no decide remediaciones, no genera código libre y no ejecuta transformaciones. Su salida se trata como una propuesta estructurada sometida a validación estricta. Esta decisión técnica diferencia AURA de asistentes que combinan diagnóstico, decisión y código en una única respuesta difícil de auditar.

La segunda contribución es experimental. AURA conserva el modelo solicitado y el observado, la configuración de inferencia, los hashes del prompt y de la entrada, la respuesta y el recibo. El Laboratorio reutiliza el mismo pipeline diagnóstico y agrega 27 ejecuciones en una matriz $3 \times 3 \times 3$. Las puntuaciones se calculan por código contra un oráculo congelado; no se emplea otro LLM como juez. La tercera contribución es operativa: un resultado de campaña puede traducirse en una configuración exacta del pipeline normal, con la advertencia de que el resultado depende del dataset y del hardware.

La memoria se organiza en diez capítulos. Tras la introducción se presenta el estado del arte; luego se describen objetivos, metodología y marco normativo. Los capítulos 5 y 6 documentan el producto, la arquitectura, los datos y la verificación. Los capítulos 7 y 8 exponen el diseño experimental, los resultados y la discusión. Finalmente, los capítulos 9 y 10 sintetizan conclusiones, limitaciones y trabajo futuro. Los anexos preservan parámetros, artefactos, glosario y trazabilidad.

2. Contexto y estado del arte

2.1. Contexto del problema

En el panorama tecnológico actual, el éxito de los sistemas de inteligencia artificial y analítica de datos depende de la integridad de la información que consumen. Los datasets pueden contener nulos, duplicados, formatos inválidos, columnas ambiguas, categorías raras, valores extremos o inconsistencias de negocio. Estos defectos no siempre impiden que un sistema funcione; muchas veces permiten que el flujo continúe, pero degradan silenciosamente la calidad de los reportes, modelos y decisiones.

Las herramientas actuales de calidad del dato suelen cubrir una parte del problema. Algunas son muy útiles para limpieza manual, otras para validación con reglas, y otras para observabilidad de pipelines. Sin embargo, pocas integran en un mismo flujo la auditoría reproducible, la interpretación asistida por modelos de lenguaje, la privacidad operativa y la decisión humana antes de ejecutar transformaciones.

2.2. Estado del arte

La inteligencia artificial centrada en datos plantea que la mejora de los sistemas de IA no depende únicamente de ajustar modelos, sino de mejorar sistemáticamente los datasets. Bajo esta mirada, la auditoría de calidad no es un trámite inicial, sino un ciclo continuo de observación, corrección y evaluación. AURA se alinea con este enfoque al priorizar la evidencia del dataset antes de cualquier interpretación asistida.

La norma ISO/IEC 25012 ofrece un marco conceptual para hablar de calidad del dato en términos de exactitud, completitud, consistencia, actualidad, trazabilidad y otras dimensiones. En AURA, estas dimensiones se traducen en reglas, hallazgos, severidades y reportes que permiten describir el estado inicial del dataset de forma reproducible.

Los modelos de lenguaje aportan una capacidad valiosa: pueden explicar hallazgos técnicos en lenguaje más comprensible y priorizar problemas. Sin embargo, también pueden producir conclusiones no soportadas, inventar columnas o sugerir acciones inapropiadas. Por esta razón, AURA no permite que el modelo actúe libremente. El diagnóstico asistido se alimenta

de un paquete de evidencia controlado y la remediación se gobierna por reglas y revisión humana.

2.3. Comparativa de herramientas frente a AURA

Herramienta / enfoque	Fortaleza principal	Limitación observada	Diferencia de AURA
OpenRefine	Limpieza exploratoria e interacción manual	Depende de intervención manual y no está centrada en diagnóstico asistido	AURA genera hallazgos reproducibles, explicación asistida y evidencia exportable.
Great Expectations	Validación robusta mediante reglas y expectativas	Requiere configuración técnica y reglas previamente definidas	AURA combina reglas deterministas con interpretación asistida y revisión humana.
Monte Carlo / observabilidad	Monitoreo de linaje, anomalías y data downtime	Orientación cloud y menor foco en explicación local del hallazgo	AURA prioriza procesamiento local-first y evidencia comprensible para el usuario.
Soda Core	Validación en pipelines CI/CD	Menor cobertura semántica y dependencia de checks declarativos	AURA añade diagnóstico asistido, scripts revisables y control humano.
AURA	Auditoría local-first con evidencia determinista y diagnóstico asistido	Benchmark multimodelo aún experimental	Integra privacidad, trazabilidad, revisión humana y laboratorio comparativo.

Tabla 1. Comparativa conceptual entre herramientas de calidad del dato y AURA.

La brecha identificada no consiste en la ausencia de herramientas, sino en la falta de un flujo integrado que preserve la reproducibilidad de las reglas, restrinja el diagnóstico asistido, mantenga trazabilidad de referencias y exija decisión humana antes de ejecutar cambios.

2.4. Inferencia local, modelos compactos y cuantización

La ejecución local de modelos de lenguaje constituye una línea relevante para AURA porque permite separar la auditoría determinista del uso de servicios externos. El procesamiento local no convierte automáticamente al sistema en privado o seguro, pero reduce la superficie de exposición cuando el diagnóstico se ejecuta en el equipo del usuario y el dataset crudo permanece en el navegador. Esta propiedad resulta especialmente valiosa en contextos donde los datos contienen información administrativa, financiera, territorial o personal.

El experimento final adopta modelos compactos porque el objetivo no es reproducir un benchmark de modelos de frontera, sino evaluar si distintas familias abiertas pueden cumplir

un contrato diagnóstico común bajo una restricción realista de hardware. La máquina de referencia es un MacBook Air M4 con 16 GB de memoria unificada y ejecución secuencial de un único modelo. Por ello, se seleccionaron Qwen3.5 4B, Gemma 4 E4B IT QAT y SmolLM3 3B, tres familias con tamaños, arquitecturas y procesos de entrenamiento diferentes, pero ejecutables mediante Ollama y GGUF.

Qwen3.5 4B representa una alternativa compacta con énfasis en seguimiento de instrucciones, cobertura multilingüe y arquitectura híbrida eficiente. Gemma 4 E4B IT QAT incorpora una familia distinta y una variante preparada para cuantización, lo que permite observar el comportamiento de un modelo con entrenamiento y representación diferentes. SmolLM3 3B funciona como línea base de menor huella, con soporte nativo para español, contexto largo y documentación abierta de entrenamiento. La selección es intencional y no presupone que uno de ellos sea superior.

La cuantización de pesos a cuatro bits disminuye memoria y coste de inferencia. Trabajos como GPTQ y AWQ muestran que la cuantización de baja precisión puede hacer viables modelos grandes en hardware más limitado, aunque la reducción de precisión numérica introduce una fuente adicional de variación que debe reconocerse como amenaza a la validez (Frantar et al., 2022; Lin et al., 2023). En AURA se mantiene el mismo preset UD-Q4_K_XL para los tres modelos con el propósito de reducir diferencias operativas atribuibles al formato de despliegue.

2.5. Riesgos de la generación directa de código mediante LLM

La generación directa de código ejecutable por un modelo de lenguaje ofrece velocidad, pero introduce riesgos incompatibles con una herramienta de auditoría gobernada. El primer riesgo es la no determinación: la misma instrucción puede producir implementaciones diferentes, dificultando repetir el experimento y demostrar qué transformación fue aprobada. El segundo riesgo es semántico: el modelo puede confundir nombres de columnas, inferir reglas de negocio inexistentes o aplicar una imputación que modifica el significado del dato.

El tercer riesgo es de seguridad. La literatura ha encontrado vulnerabilidades en código generado por asistentes, incluso cuando el código parece funcional. Pearce et al. (2021) identificaron una proporción importante de programas vulnerables en escenarios asociados

a debilidades de alto riesgo. Además, los modelos pueden inventar dependencias o paquetes inexistentes; Spracklen et al. (2024) documentaron este fenómeno a gran escala y mostraron su relevancia para la cadena de suministro de software.

El cuarto riesgo es de autoridad y trazabilidad. Si el LLM diagnostica, decide la remediación, redacta el código y lo ejecuta, el sistema concentra demasiada autonomía en un componente probabilístico. Una explicación convincente podría convertirse en una operación destructiva sin una frontera clara entre evidencia, recomendación y ejecución. AURA reduce ese riesgo al limitar al modelo a la interpretación estructurada y trasladar la decisión, la generación del script y la ejecución a componentes verificables y supervisados.

2.6. Evaluación reproducible de modelos de lenguaje

La evaluación de modelos de lenguaje no puede reducirse a una única exactitud. HELM propone cobertura explícita, medición multidimensional y transparencia sobre lo que queda fuera de la evaluación (Liang et al., 2022). AURA adopta esa orientación en escala local: combina alineación con ground truth, fiabilidad, cumplimiento contractual, soporte de evidencia, claims no soportados, latencia y tokens. Cada dimensión conserva su denominador; los fallos no se eliminan ni se reemplazan por cero cuando una métrica no es aplicable.

El ground truth cumple una función de control: permite comprobar si la respuesta cubre el registro canónico de hallazgos. Sin embargo, como el contrato ya exige esa cobertura, precisión, recall y F1 no reciben peso en el índice equilibrado de la campaña definitiva. De lo contrario se premiaría dos veces una misma condición. El contrato actúa como gate de validez y el índice pondera fiabilidad, evidencia, seguridad frente a claims sin soporte y eficiencia.

Esta distinción es importante para interpretar un F1 de 1,0. En la campaña no significa que el modelo descubrió por sí solo todos los defectos posibles del dataset, sino que reprodujo correctamente el registro conocido que debía cubrir. La utilidad adicional se observa en la calidad de las explicaciones, el anclaje a referencias visibles, la ausencia de afirmaciones no soportadas y la estabilidad entre repeticiones.

2.7. Brecha de investigación y posición de AURA

La literatura de calidad del dato describe dimensiones y procedimientos de medición; las herramientas de validación automatizan reglas; los LLM facilitan explicaciones; y los enfoques local-first reducen la dependencia de servicios centrales. La brecha abordada por AURA aparece en la intersección: se necesita un flujo donde un hecho determinista pueda transformarse en una explicación útil, una decisión humana y una corrección reproducible sin perder el origen de cada afirmación.

AURA no pretende reemplazar OpenRefine, Great Expectations, Soda u observabilidad empresarial. Su aporte es un prototipo de integración gobernada para el análisis de un CSV: evidencia mínima, contrato estructurado, separación de autoridad, ejecución sobre copia y expediente exportable. En el Laboratorio, esa misma arquitectura se convierte en una guía para elegir una combinación de modelo y entrada para un dataset controlado. La recomendación es contextual y reversible, no una clasificación universal de modelos.

La propuesta es coherente con el modelo de calidad de datos de ISO/IEC 25012:2008, con la evaluación contextual defendida por Wang y Strong (1996) y Pipino, Lee y Wang (2002), y con los principios de trazabilidad y gestión de riesgo de NIST AI RMF 1.0. La novedad no radica en cada componente aislado, sino en el contrato que limita su autoridad y conserva evidencia para auditar el conjunto.

3. Objetivos concretos y metodología de trabajo

3.1. Objetivo general

Consolidar AURA como una arquitectura local-first, reproducible y evaluable para auditoría inteligente de calidad del dato, capaz de generar evidencia determinista, restringir el diagnóstico asistido mediante paquetes de evidencia, producir planes de remediación gobernados, habilitar revisión humana y preparar una ejecución controlada y trazable.

3.2. Objetivos específicos

1. Consolidar una arquitectura local-first que cargue, procese y audite datasets CSV en el navegador, generando trazas y minimizando la información compartida con proveedores externos.
2. Diseñar y evaluar un motor determinista basado en reglas explícitas, heurística de tipos y estadística descriptiva, documentando verdaderos positivos, falsos positivos, falsos negativos, precisión, recall y F1.
3. Implementar una capa de diagnóstico asistido restringida a evidencia estructurada, evitando que el modelo opere sobre el dataset completo o genere transformaciones sin control.
4. Implementar un laboratorio de comparación de modelos bajo un contrato común, diferenciando claramente pruebas operativas, resultados experimentales y benchmark formal.
5. Implementar gobernanza human-in-the-loop mediante revisión y aprobación humana antes de generar o ejecutar cualquier script de limpieza.
6. Preparar scripts Python/Pandas revisables a partir de acciones aprobadas, manteniendo trazabilidad entre hallazgos, decisiones y código propuesto.

3.3. Metodología del trabajo

El trabajo sigue una metodología de desarrollo de software orientada por evidencia. CRISP-DM aporta la secuencia iterativa de comprensión del problema, comprensión de datos, preparación, modelado, evaluación y despliegue (Chapman et al., 2000). Scrum se utiliza como marco ligero para organizar incrementos verificables y retroalimentación, sin afirmar

que el proyecto individual reproduzca todos los roles de un equipo Scrum (Schwaber & Sutherland, 2020). La evaluación experimental se diseña después de estabilizar los contratos del producto, de modo que los resultados midan el sistema que realmente utiliza la persona.

El desarrollo se dividió en incrementos: motor determinista y perfilamiento; contrato de evidencia y diagnóstico; informe y exportación; plan de remediación; renderer de script; revisión humana; runner y recibo; reauditoría; y Laboratorio. Cada incremento se cerró con pruebas focalizadas, typecheck, build y, cuando el recorrido lo exigía, Playwright. Esta secuencia permitió detectar fallos de integración que no aparecían en pruebas unitarias, como diferencias entre el pipeline normal y el runner experimental, truncamientos de salida o estados de interfaz que ocultaban el avance.

Se emplean tres clases de evidencia. La evidencia de producto demuestra que una persona puede completar el flujo; la evidencia experimental compara modelos bajo un protocolo congelado; y la evidencia académica vincula decisiones con normas y literatura. Ninguna clase sustituye a las otras. Un build correcto no demuestra usabilidad; una campaña formal no garantiza generalización; y una explicación académica no reemplaza el recibo de una ejecución.

3.4. Preguntas de investigación y variables

RQ1 evalúa la capacidad del motor determinista para activar reglas conocidas. Sus variables son TP, FP, FN, precisión, recall y F1, calculadas sobre una anotación explícita. RQ2 evalúa el diagnóstico LLM. Sus variables son cobertura exacta de ruleId + columnId + scope, cumplimiento de JSON V2, referencias de evidencia, claims sin soporte, latencia, tokens y estabilidad. RQ3 evalúa la remediación: identidad del script, validación sintáctica, recibo, hashes del CSV y delta de reauditoría. RQ4 compara combinaciones modelo-entrada mediante dimensiones separadas y un índice equilibrado documentado.

Las variables se calculan de forma determinista. AURA no utiliza un LLM como juez ni asigna puntuaciones por estilo narrativo. La revisión humana de claridad, trazabilidad y accionabilidad se conserva como posibilidad de evaluación cualitativa, pero no bloquea ni puntúa las 27 corridas de la campaña definitiva. Esta decisión reduce carga manual y separa la evaluación del modelo del control humano que gobierna una remediación real.

Los fallos son observaciones, no datos faltantes que deban ocultarse. Una salida truncada, un JSON inválido, una referencia inexistente o un modelo observado diferente del solicitado reducen la fiabilidad de la celda. Cuando una métrica no puede calcularse, se informa como no disponible; no se imputa un cero salvo en la puntuación de una celda sin corridas válidas, donde el cero expresa que no existe evidencia utilizable para recomendarla.

3.5. Diseño de las campañas y congelación del protocolo

El diseño experimental utiliza un dataset controlado con ground truth, tres modelos, tres métodos de entrada y tres repeticiones por combinación. La matriz contiene 27 diagnósticos evaluados. Antes de cada bloque modelo-repetición se realiza un calentamiento que se excluye del análisis, para un total de nueve warm-ups y 36 llamadas reales. El orden de las unidades se conserva en campaign.json, junto con el snapshot de entorno, los modelos, la configuración y los hashes.

Una campaña solo es formal cuando todas las unidades fueron intentadas, los recibos son coherentes, los modelos observados corresponden a los solicitados y los artefactos de exportación pasan los guards. Congelar el protocolo evita cambiar el prompt, el esquema, el oráculo o las ponderaciones después de observar resultados. La primera campaña se trata como piloto de calibración porque reveló decisiones de producto y evaluación que debían corregirse; la segunda utiliza el protocolo 2.6.0 y constituye el experimento definitivo de esta memoria.

Tabla 1. Correspondencia entre preguntas, métricas y evidencia

Pregunta	Métrica principal	Artefacto verificable	Límite
RQ1 Motor	Precisión, recall y F1	Ground truth y activaciones	Solo reglas anotadas
RQ2 Diagnóstico	Contrato, evidencia, claims, fiabilidad	Respuesta, snapshot y recibo	Un dataset controlado
RQ3 Remediación	Sintaxis, hashes y delta	Script, bundle, receipt, corrected.csv	Una copia y decisiones aprobadas
RQ4 Selección	Dimensiones e índice equilibrado	results-summary.json	No ganador universal

Elaboración propia.

3.6. Integridad académica, reproducibilidad y uso de IA

La reproducibilidad se apoya en artefactos y no en memoria del proceso. Cada resultado importante se vincula con un archivo exportado, una versión del protocolo, un hash o una captura. Las cifras de la campaña se leen desde results-summary.json y report.md; las

imágenes de la interfaz no se emplean para reconstruir manualmente métricas. El código, las pruebas y las salidas forman un expediente coherente.

Para reducir riesgo de similitud y atribución incorrecta, la redacción final sintetiza las fuentes con lenguaje propio, conserva citas autor–fecha y evita reproducir extensamente textos normativos o documentación. Turnitin debe interpretarse como una señal de revisión, no como prueba automática de autoría o de ausencia de asistencia. La responsabilidad final permanece en el autor.

4. Marco normativo

4.1. Protección de datos de carácter personal

Los archivos auditados pueden contener nombres, identificadores, direcciones, registros administrativos, información financiera o datos de salud. Cuando esos valores identifican o hacen identificable a una persona, resultan aplicables el Reglamento General de Protección de Datos y la Ley Orgánica 3/2018. El artículo 5 del RGPD establece, entre otros, los principios de licitud, transparencia, limitación de finalidad, minimización, exactitud, limitación del plazo e integridad y confidencialidad (Reglamento [UE] 2016/679).

AURA no determina por sí sola la base jurídica del tratamiento ni sustituye una evaluación de impacto. Su contribución es técnica: procesa el CSV en el navegador, reduce el contenido enviado al modelo, marca patrones sensibles y conserva una cadena de decisiones. El usuario sigue siendo responsable de decidir si puede tratar el dataset, qué muestras son necesarias y con quién puede compartir el expediente exportado.

El CSV corregido puede conservar información personal. Por ello, el paquete de evidencia excluye el dataset fuente y advierte cuando incluye `corrected.csv`. Los hashes no anonimizan: solo permiten comprobar identidad e integridad. Un hash de un valor de baja entropía tampoco debe tratarse como anonimización irreversible.

4.2. Minimización y procesamiento local

El principio local-first describe software que conserva la copia primaria de los datos y permite que el usuario trabaje sin depender permanentemente de un servidor central (Kleppmann et al., 2019). En AURA, el CSV se carga, perfila y audita en el navegador. La inferencia puede ejecutarse con Ollama en localhost, mientras que los proveedores externos son opcionales y quedan sujetos a la configuración del usuario.

Local-first no equivale a seguridad automática. El navegador, el sistema operativo, las extensiones, el almacenamiento local y los archivos exportados siguen siendo superficies de riesgo. AURA aplica minimización al diagnóstico: envía un envelope con metadatos, reglas y muestras limitadas, nunca el dataset completo. Los tres métodos de entrada aumentan

progresivamente el contexto visible, y el recibo certifica cuál fue solicitado y cuál resultó efectivo.

La minimización también es experimental. Contexto mínimo puede ser suficiente para cumplir el contrato y reducir superficie de exposición; Evidencia equilibrada y Evidencia completa aportan más señales, pero pueden aumentar longitud, latencia y oportunidades de formular claims no soportados. La campaña evalúa ese compromiso en lugar de asumir que más contexto produce siempre mejor diagnóstico.

4.3. Ética en IA y gobernanza

NIST AI RMF 1.0 organiza la gestión de riesgo de IA alrededor de las funciones gobernar, mapear, medir y gestionar (Tabassi, 2023). AURA materializa esas funciones a escala de prototipo: define contratos y autoridad, mapea hallazgos a evidencia, mide cumplimiento y conserva mecanismos para bloquear o escalar decisiones. El perfil de IA generativa de NIST añade riesgos como confabulación, privacidad y seguridad de la información, relevantes cuando un modelo interpreta muestras de datos (Autio et al., 2024).

El Reglamento de IA de la Unión Europea subraya transparencia, documentación, registros y supervisión humana para sistemas de alto riesgo (Reglamento [UE] 2024/1689). AURA no se clasifica en esta memoria como sistema de alto riesgo ni como producto conforme al Reglamento; se emplean esos principios como referencia de diseño. La revisión humana se concentra donde existe autoridad para modificar datos, no en cada salida experimental del Laboratorio.

La gobernanza determinista corrige una asimetría: el modelo puede redactar un valor `requiresHumanReview` incorrecto, pero no puede rebajar la política del producto. AURA conserva la respuesta cruda para el Laboratorio y aplica la política efectiva en el pipeline normal, dejando evidencia de la normalización. Esta intervención solo modifica el campo de revisión; no repara JSON, identificadores, cobertura ni claims.

4.4. Integridad criptográfica, trazabilidad y responsabilidad humana

La gobernanza de AURA no se limita a mostrar una pantalla de aprobación. La arquitectura conserva una cadena de identidad entre el paquete de entrada del diagnóstico, la respuesta del modelo, el plan de remediación, el script aprobado, el dataset fuente, el recibo de

ejecución y el CSV resultante. Esta cadena se materializa mediante referencias y hashes SHA-256.

El objetivo de los hashes no es ocultar información ni sustituir controles de seguridad, sino detectar alteraciones y asociaciones incorrectas. Un recibo válido debe corresponder al mismo script aprobado, al mismo dataset fuente y al mismo bundle preparado por AURA. Del mismo modo, el hash del CSV resultante debe coincidir con el valor declarado por el runner antes de permitir la reauditoría.

La persona mantiene la responsabilidad sobre la aprobación. El sistema puede impedir estados incoherentes, pero no conoce todas las reglas de negocio. Por esta razón, AURA combina automatización verificable con revisión humana: automatiza la comprobación de identidades, contratos y sintaxis, mientras la decisión sobre el significado y la conveniencia de una transformación permanece bajo control humano.

4.5. Integridad criptográfica y trazabilidad

AURA utiliza SHA-256 para vincular dataset, prompt, entrada, respuesta, script y salida. FIPS 180-4 define SHA-256 como algoritmo para producir un resumen capaz de detectar cambios en un mensaje (NIST, 2015). Para que el hash de un objeto JSON sea repetible, la aplicación utiliza una serialización canónica estable; el principio coincide con RFC 8785, que exige una representación invariante antes de firmar o hashear JSON (Rundgren et al., 2020).

Un hash no demuestra que el contenido sea correcto ni seguro. Demuestra que dos artefactos comparados son idénticos bajo la misma función. Por ello, el recibo combina identidad criptográfica con validaciones semánticas: modelo observado, estado de sintaxis, dimensiones del CSV, códigos de error y referencia al bundle. La reauditoría vuelve a ejecutar reglas; no infiere calidad a partir del hash.

Los contratos JSON V2 se alinean con la finalidad de JSON Schema Draft 2020-12: describir estructura y validar instancias. Ollama admite salidas estructuradas mediante un esquema JSON, pero la documentación también recomienda instrucciones explícitas y baja temperatura para mejorar consistencia. AURA añade un validador propio porque una salida que parsea como JSON todavía puede contener referencias inválidas o claims no soportados.

5. Desarrollo específico de la contribución

Este capítulo describe la contribución como trabajo de desarrollo de software. La explicación se organiza de forma gradual: primero se presentan requisitos, después se definen los componentes técnicos en lenguaje académico, luego se muestra la arquitectura y finalmente se incorporan capturas y fragmentos de código explicados.

5.1. Identificación de requisitos

El problema que guía AURA es el deterioro silencioso de la calidad del dato antes de su uso en reportes, modelos o procesos de decisión. El usuario objetivo es un analista, ingeniero de datos, auditor técnico o equipo de datos que necesita comprender la salud de un CSV sin enviar el archivo completo a un servidor externo.

5.1.1. Requisitos funcionales

- Cargar archivos CSV localmente desde el navegador.
- Perfilar el dataset mediante reglas deterministas.
- Presentar hallazgos con severidad, columna afectada, conteo y evidencia.
- Construir un paquete estructurado de evidencia para el diagnóstico asistido.
- Generar un plan de acciones de remediación bajo políticas controladas.
- Permitir aprobación o rechazo humano de acciones.
- Preparar scripts Python/Pandas revisables desde acciones aprobadas.
- Documentar resultados, límites y evidencia exportable.

5.1.2. Requisitos no funcionales

- Privacidad desde el diseño: el archivo crudo se procesa localmente por defecto.
- Reproducibilidad: los hallazgos deben depender de reglas explícitas y no de interpretación probabilística.
- Trazabilidad: cada etapa debe indicar qué recibe, qué produce y qué evidencia deja.
- Gobernanza humana: ninguna acción destructiva debe avanzar sin aprobación.
- Claridad: los términos técnicos deben explicarse antes de usarse en la memoria.

5.2. Diccionario funcional de componentes de AURA

Para evitar que el documento dependa de nombres internos del repositorio, la Tabla 2 traduce cada componente técnico a un concepto académico y operativo. A partir de esta tabla, los nombres técnicos se usan únicamente como referencia complementaria.

Nombre técnico	Nombre académico	Función	Riesgo que controla
csvService.ts	Módulo de lectura local	Lee y convierte el CSV en datos utilizables dentro del navegador.	Evita depender de un servidor para iniciar el análisis.
auditEngine.ts	Motor determinista de auditoría	Aplica reglas explícitas para detectar nulos, duplicados, espacios invisibles, formatos inválidos y otros patrones.	Reduce variabilidad y permite repetir el mismo análisis sobre el mismo archivo.
executionEvidence.ts	Registro de evidencia de ejecución	Guarda metadatos del análisis: archivo, columnas, tiempos, fingerprint y trazas.	Permite explicar de dónde salió cada resultado.
AuditReport	Informe técnico de auditoría	Resume hallazgos, severidad, columnas afectadas, conteos y score de salud.	Evita diagnósticos vagos o puramente narrativos.
EvidenceEnvelopeV2	Paquete estructurado de evidencia	Empaqueta solo la evidencia necesaria para el diagnóstico asistido.	Evita entregar el dataset completo al modelo y reduce alucinaciones por falta de contexto.
DiagnosisResponseV2	Diagnóstico asistido estructurado	Organiza la interpretación del modelo de lenguaje en campos verificables.	Reduce respuestas libres, columnas inventadas y conclusiones sin soporte.
RemediationPlanV2	Plan de remediación gobernado	Convierte hallazgos en acciones posibles, clasificadas por riesgo y revisables por el usuario.	Evita que el modelo autorice transformaciones por sí solo.
ScriptContractV2	Contrato verificable de script	Representa qué acciones aprobadas se convierten en código Python/Pandas y cómo se verifica su integridad.	Evita scripts no trazables o modificados sin control.
Renderer determinista	Generador controlado de código	Produce código desde acciones aprobadas, no desde texto libre del modelo.	Reduce riesgo de código arbitrario o no autorizado.
HITL	Revisión humana antes de aprobar	Permite aprobar o rechazar acciones antes de continuar.	Mantiene supervisión humana sobre decisiones de limpieza.
BenchmarkLab	Laboratorio de comparación de modelos	Permite comparar proveedores y modelos bajo un flujo común.	Evita seleccionar modelos solo por intuición o disponibilidad.

Tabla 2. Diccionario funcional de componentes de AURA.

5.3. Arquitectura y funcionamiento general

AURA funciona como una cadena de responsabilidades. Primero, el archivo se carga localmente. Después, el motor determinista produce un informe técnico. Luego, se construye un paquete de evidencia reducido para el diagnóstico asistido. A partir de esa interpretación, el sistema prepara un plan de remediación; la persona revisa y aprueba acciones; finalmente, las acciones aprobadas pueden convertirse en un script revisable.

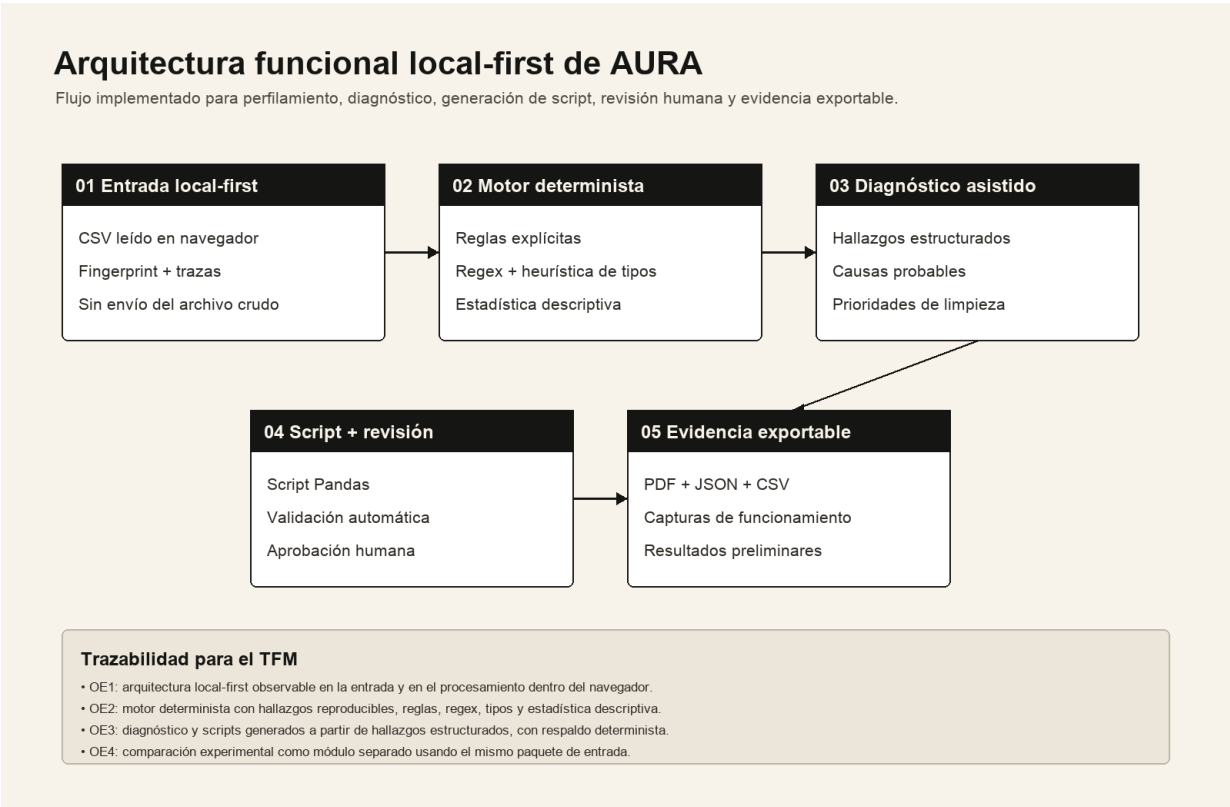
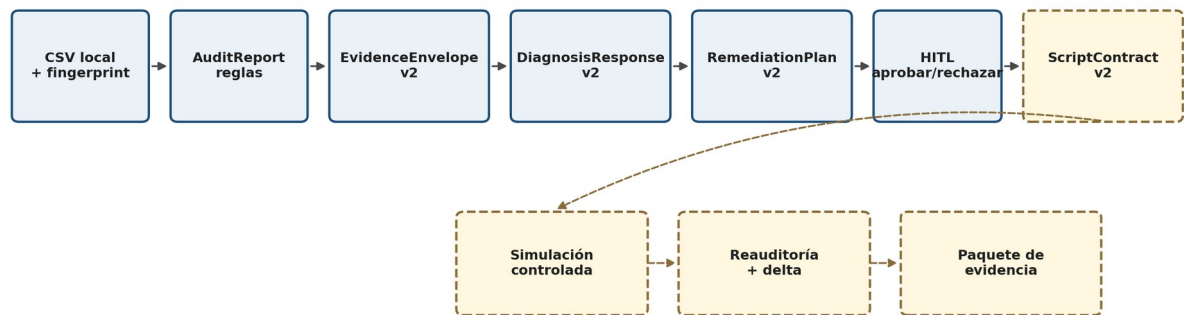


Figura 1. Arquitectura funcional local-first de AURA.

La Figura 1 resume la primera arquitectura funcional de AURA. En ella se observa una secuencia clara: entrada local, motor determinista, diagnóstico asistido, script con revisión y evidencia exportable. Esta arquitectura ya evita enviar el archivo crudo como requisito de funcionamiento y sitúa la revisión humana como parte del flujo.

Arquitectura actual de AURA basada en contratos verificables

Línea continua: implementado y auditado. Línea discontinua: siguiente cierre técnico.



Principio de seguridad: el LLM interpreta evidencia; la política determinista decide; la persona aprueba; el renderer ejecuta.

Figura 2. Arquitectura actual basada en evidencia, diagnóstico, decisión y ejecución controlada.

La Figura 2 representa la evolución conceptual de la herramienta. La diferencia más importante es que AURA ya no se describe como un flujo lineal donde el modelo produce directamente un script. Ahora se plantea como una arquitectura por etapas: evidencia, diagnóstico, remediación, revisión humana y preparación de ejecución controlada. Los bloques discontinuos representan actividades que se completarán en la fase final: ejecución segura, reauditoría y comparación antes/después.

5.3.1. Flujo operativo actual de AURA

En su estado actual, AURA presenta al usuario una secuencia de trabajo explícita: Carga, Perfil, Diagnóstico, Script, Revisión y Exportar. La etapa de Carga recibe el archivo CSV sin enviarlo por defecto a un servidor. La etapa de Perfil resume dimensiones, columnas y hallazgos preliminares del dataset. La etapa de Diagnóstico interpreta dichos hallazgos y propone hipótesis y acciones en lenguaje comprensible. Luego, la etapa de Script prepara una propuesta revisable de limpieza en Python/Pandas a partir de las acciones aprobables. La etapa de Revisión exige intervención humana antes de aprobar la propuesta, y la etapa de Exportar organiza la salida del proceso en forma de script, evidencia y artefactos de seguimiento.

5.3.2. Proveedor local y adaptadores de diagnóstico

El recorrido principal validado para esta memoria utiliza Ollama Local. AURA consulta `/api/tags` para detectar los modelos instalados y propaga la lista al diagnóstico y al Laboratorio. La disponibilidad del endpoint no basta para afirmar que un modelo concreto se

ejecutó: cada recibo conserva modelo solicitado, modelo observado y, cuando está disponible, digest local.

La arquitectura mantiene adaptadores para otros proveedores, pero el experimento formal se congela sobre Ollama para reducir variación de red, preservar inferencia local y capturar parámetros homogéneos. La separación por adaptadores permite ampliar proveedores sin modificar el contrato diagnóstico ni el evaluador, siempre que el adaptador reporte la respuesta cruda y las métricas requeridas.

La lista de modelos no se codifica como catálogo estático. Se obtiene del runtime local, evitando mostrar opciones no instaladas. En la campaña 2 se verificaron Qwen3.5 4B, Gemma 4 E4B y SmolLM3 3B con cuantización UD-Q4_K_XL. La configuración observada fue temperature 0,1, top_p 0,9, num_ctx 32.768, num_predict 8.192, think desactivado, keep_alive 10 minutos y timeout 900 segundos.

5.3.3. Arquitectura funcional final

El flujo principal final se simplifica en cinco etapas visibles: Carga, Perfil, Diagnóstico, Reporte y Exportación. Desde el Diagnóstico se habilita una rama opcional de remediación. Esta rama no altera el recorrido analítico principal y solo se activa cuando la persona decide avanzar hacia una limpieza controlada.

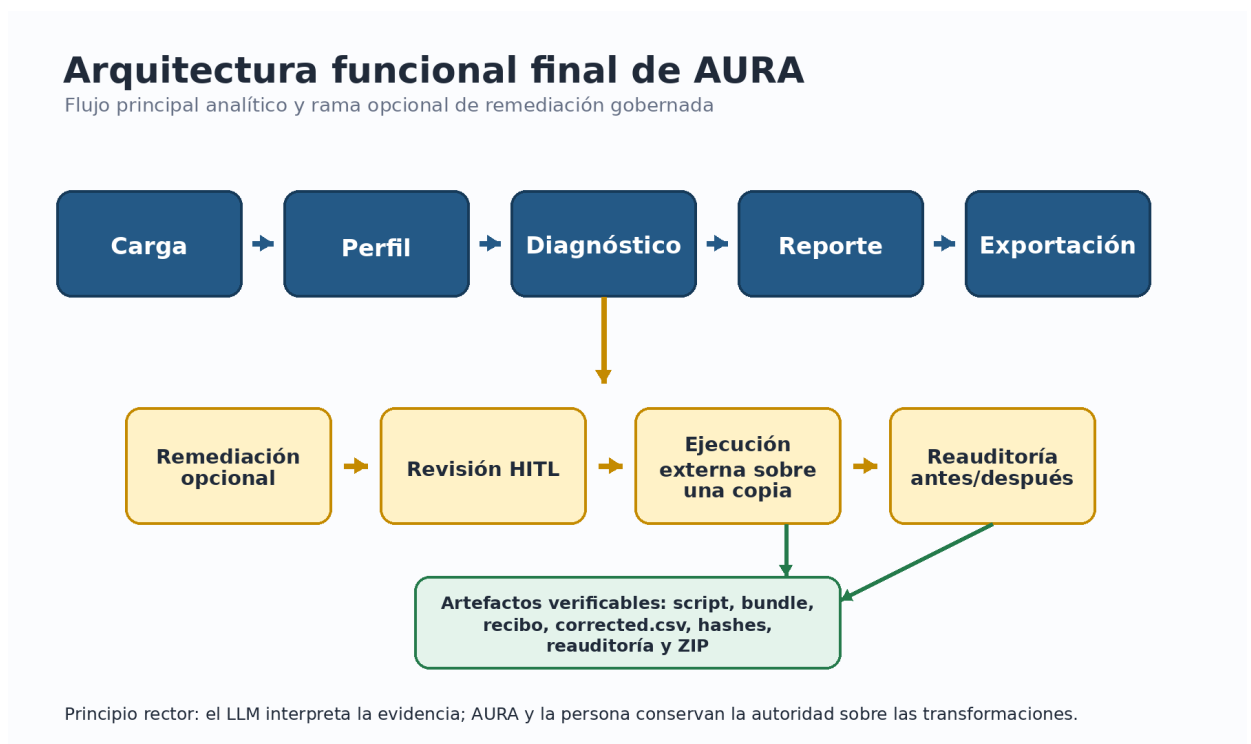


Figura 11. Arquitectura funcional final de AURA y rama opcional de remediación gobernada.

La separación entre el flujo principal y la rama opcional resuelve una tensión frecuente en herramientas de calidad del dato: analizar no implica necesariamente modificar. Un usuario puede obtener perfil, diagnóstico, reporte y exportación sin ejecutar una transformación. Si decide remediar, debe pasar por HITL, ejecución externa y reauditoría.

5.3.4. Frontera de autoridad entre el LLM, AURA y la persona

El LLM tiene autoridad interpretativa limitada. Puede organizar hallazgos, explicar riesgos y proponer prioridades dentro del contrato diagnóstico. No tiene autoridad para recalcular el score determinista, crear columnas que no existen, decidir unilateralmente una remediación ni ejecutar código. AURA conserva la autoridad técnica sobre contratos, vocabulario de acciones, registro de columnas, hashes y validaciones; la persona conserva la autoridad decisonal sobre la aprobación.

Esta frontera es una contribución central porque transforma al modelo de lenguaje en un componente reemplazable. Si un proveedor falla o incumple el contrato, el motor determinista y la evidencia original permanecen válidos. La arquitectura no depende de aceptar como cierta una respuesta narrativa del modelo.

5.3.5. Generación determinista del script Python/Pandas

AURA no solicita al modelo que redacte un programa completo. Después del diagnóstico, el sistema construye un plan de remediación con un vocabulario cerrado de acciones. Cada acción tiene identificador, columna, parámetros, nivel de riesgo y estado de aprobación. El renderer determinista traduce exclusivamente las acciones aprobadas a plantillas Python/Pandas conocidas.

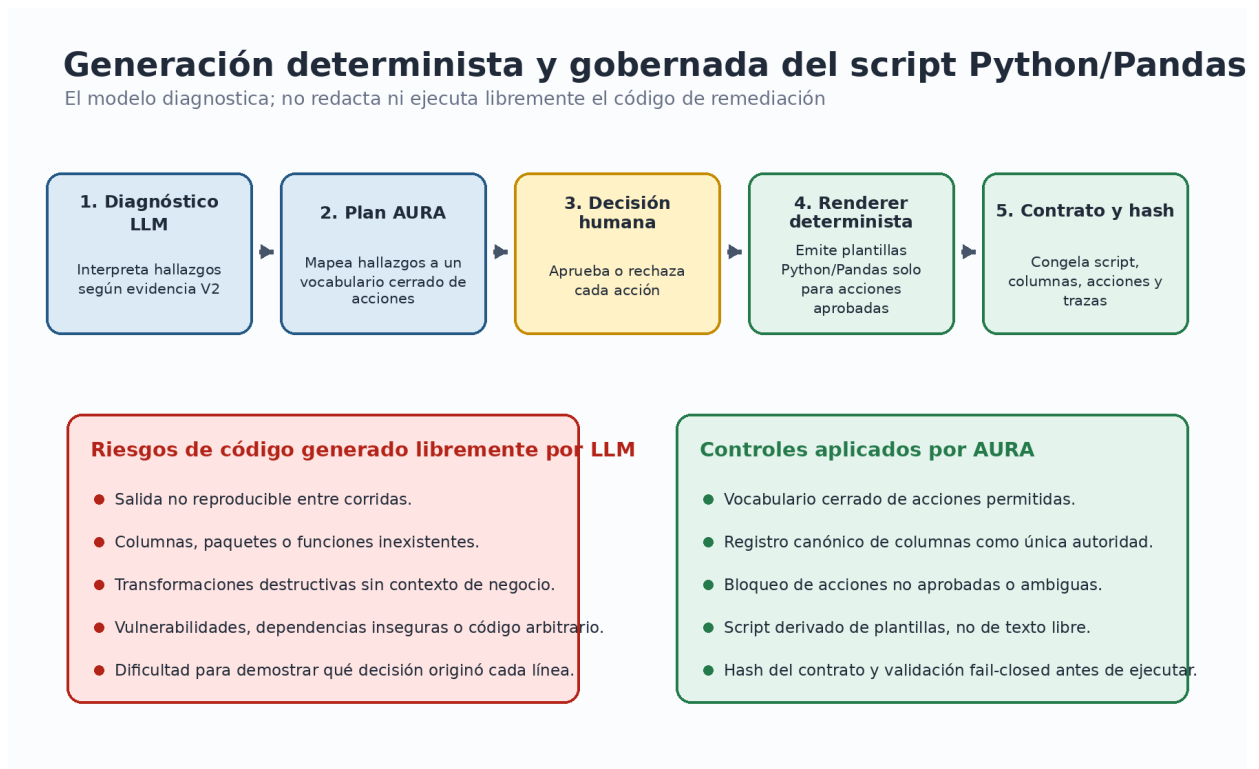


Figura 12. Generación determinista del script y controles frente a código libre del LLM.

Entre las acciones implementadas se encuentran recorte de espacios, normalización de placeholders, normalización de mayúsculas/minúsculas, conversión de números disfrazados y eliminación de duplicados exactos. Una acción no soportada, una columna ambigua o un parámetro fuera del contrato produce un error y bloquea la generación.

5.3.6. Descarga, ejecución externa y dataset corregido

Una vez aprobado el script, AURA prepara un bundle que contiene el código, el hash del script aprobado, el hash del dataset fuente y las referencias al diagnóstico y al paquete de evidencia. El bundle se descarga para ser ejecutado por un runner externo de Python/Pandas sobre una copia del CSV. Esta decisión mantiene el navegador fuera del entorno de ejecución Python y evita modificar silenciosamente el archivo fuente.

La ejecución produce dos artefactos principales: `corrected.csv` y `receipt.json`. El primero es la copia transformada que el usuario puede descargar y conservar. El segundo registra la versión de Python, la versión de Pandas, la plataforma, el estado de sintaxis, el estado de ejecución, duración, hashes de salida estándar y error, dimensiones del resultado y SHA-256 del CSV corregido.

5.3.7. Importación verificable y reauditoría

Para cerrar el ciclo, el usuario vuelve a AURA y carga el dataset fuente, el CSV corregido y el recibo. La aplicación verifica que el archivo fuente sea idéntico byte a byte al dataset congelado, que el recibo corresponda al script y al bundle aprobados, y que el hash del CSV resultante coincida con `afterDatasetSha256`. Si alguna comprobación falla, el resultado queda bloqueado y no se certifica una mejora.

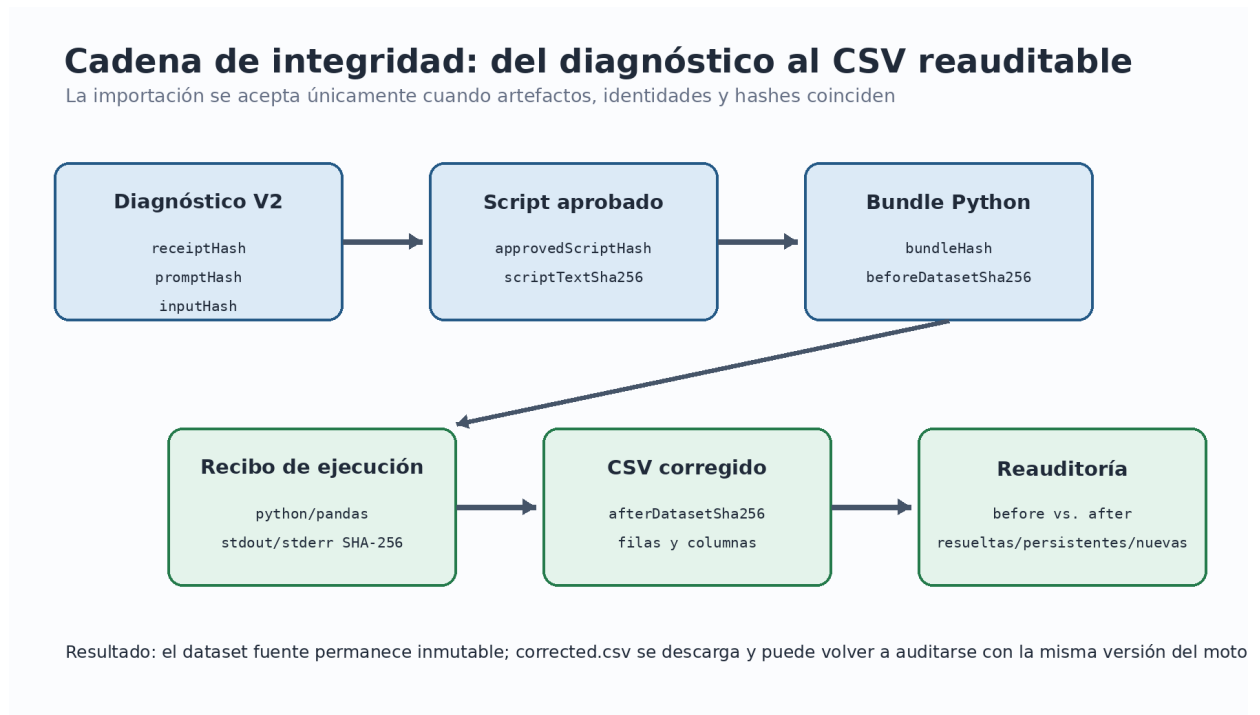


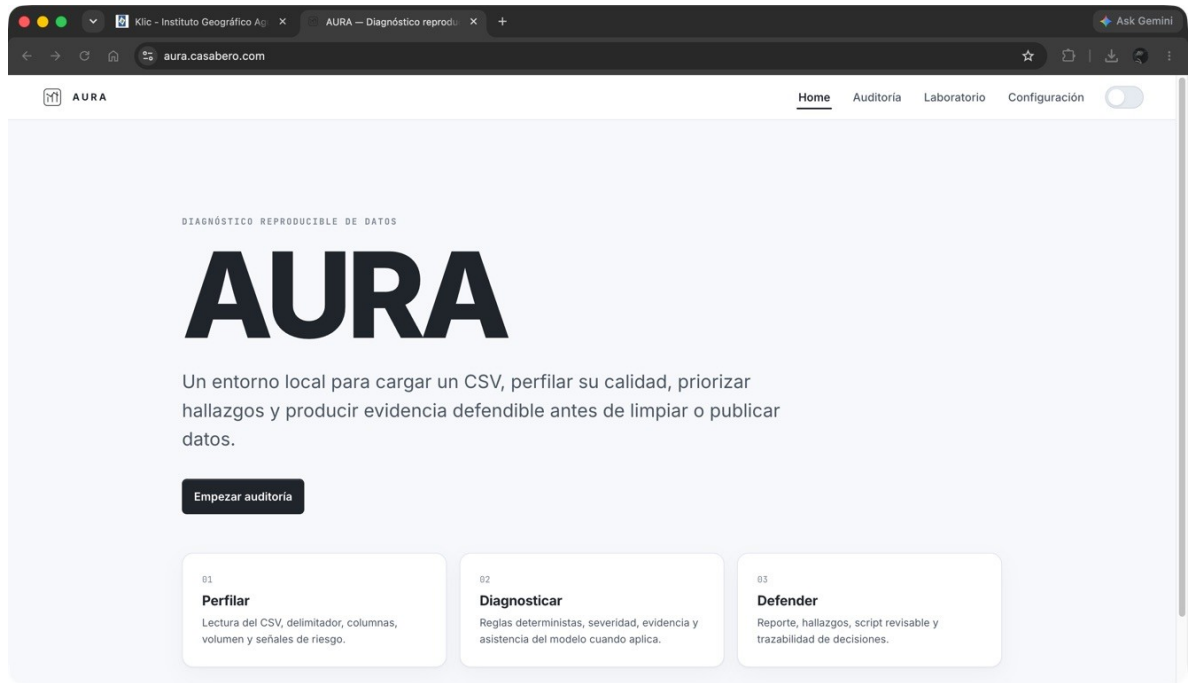
Figura 13. Cadena de integridad de la ejecución externa y la reauditoría.

Cuando la cadena es válida, AURA ejecuta de nuevo el mismo motor determinista sobre el CSV original y sobre `corrected.csv`. El resultado diferencia reglas resueltas, persistentes y nuevas, además del cambio en score, número de hallazgos, filas y columnas. Esta comparación permite afirmar una mejora técnica dentro del alcance de las reglas, sin presentar el score como una garantía universal de corrección semántica.

5.4. Evidencia visual del prototipo

Las siguientes capturas corresponden al estado publicado de AURA el 14 de julio de 2026 y a la segunda campaña. Se seleccionan vistas que demuestran funciones distintas: entrada al producto, preparación del Laboratorio, estado de las 27 unidades, exploración de resultados, traducción a configuración y verificación de una remediación. Las cifras experimentales se toman de los artefactos JSON; las capturas sirven como evidencia de interfaz.

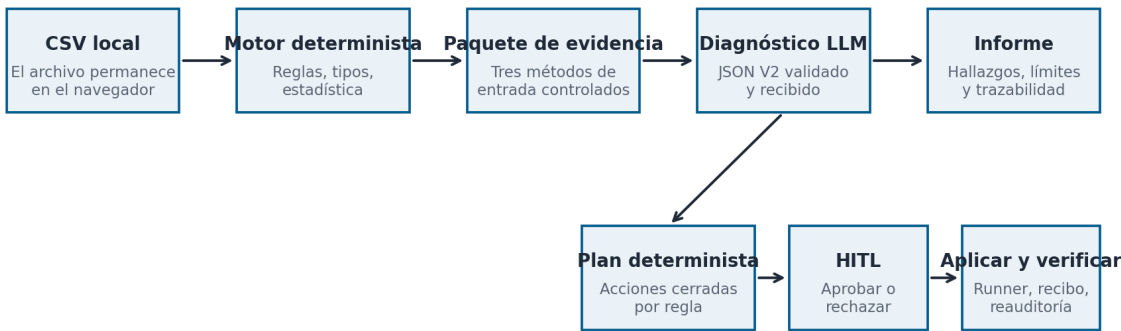
Figura 1. Página principal de AURA y propuesta de valor



Fuente: captura del prototipo publicado en aura.casabero.com, 14 de julio de 2026.

Figura 2. Arquitectura funcional y frontera de autoridad

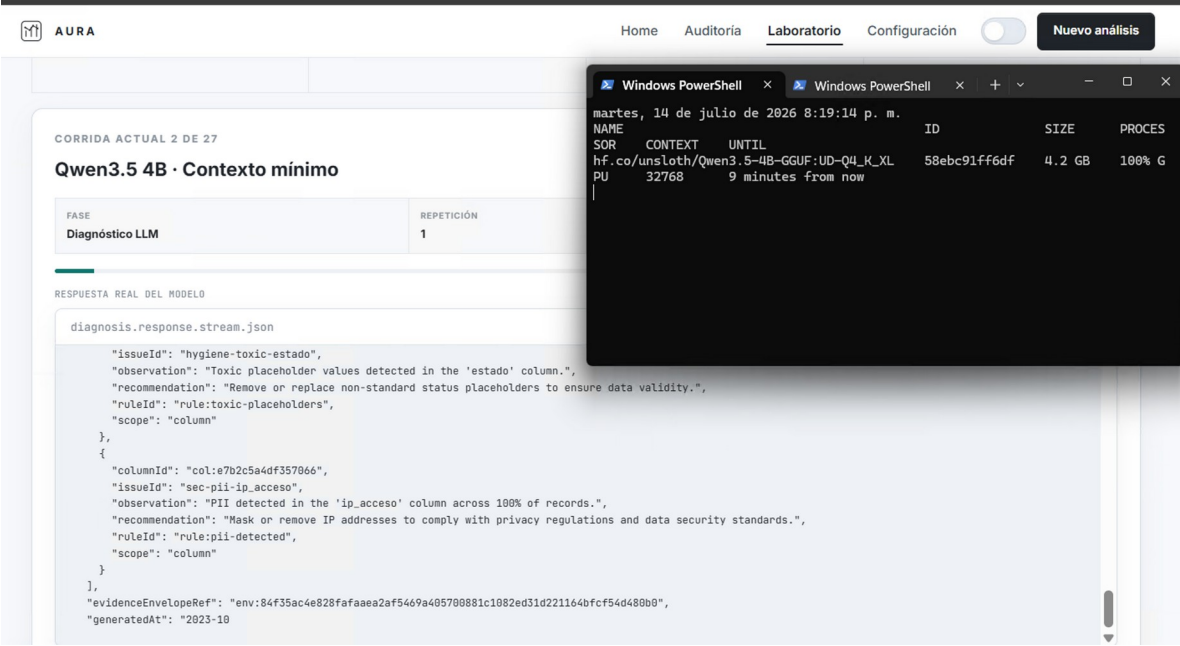
Arquitectura funcional y frontera de autoridad



El Laboratorio reutiliza el mismo diagnóstico; la rama de remediación permanece en el pipeline normal.

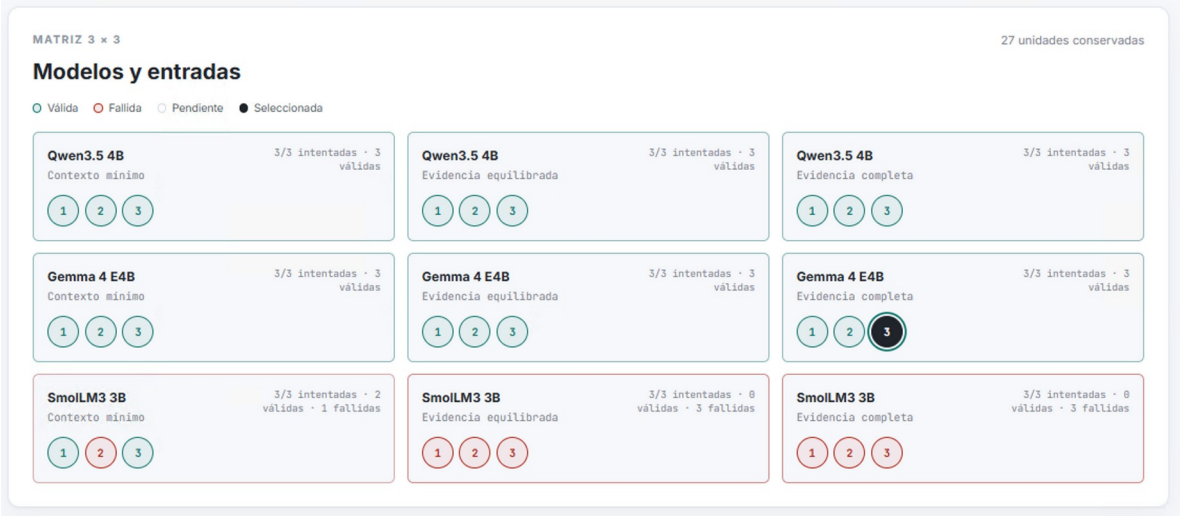
Fuente: elaboración propia a partir de la implementación de AURA.

Figura 3. Ejecución de una corrida y verificación del modelo cargado en Ollama



Fuente: AURA, campaña 2. La consola muestra el modelo realmente cargado por Ollama durante la ejecución.

Figura 4. Matriz de modelos, entradas y repeticiones de la campaña definitiva



Fuente: AURA, campaña 2, protocolo 2.6.0.

Figura 5. Traducción de un resultado del Laboratorio a parámetros del pipeline normal

DEL RESULTADO A LA PRÁCTICA

modelo disponible

Configuración para el próximo diagnóstico

La selección de la matriz y los gráficos se traduce aquí en parámetros exactos del pipeline normal.

MODELO LLAMA

hf.co/unsloth/Qwen3.5-4B-GGUF:UD-Q4_K_XL

Contexto mínimo

TEMPERATURA	TOP_P	NUMCTX	NUMPREDICT
0.1	0.9	32.768	8.192
THINK	TIMEOUT	KEEP ALIVE	SEED
No	900s	10m	automática

ÍNDICE EQUILIBRADO	FIABILIDAD	EVIDENCIA	CORRIDAS VÁLIDAS
92.6/100	100.0/100	100.0/100	3/3

Configuración observada en esta campaña y este hardware. Aplicarla no garantiza el mismo resultado con otro dataset.

Copiar configuración

Aplicar al próximo diagnóstico

Ir a Auditoría

Fuente: AURA, campaña 2. La recomendación es contextual y no garantiza el mismo resultado con otro dataset.

Figura 6. Ejecución Python verificada y reauditoría de la copia corregida

AURA

HomeAuditoríaLaboratorioConfiguración

Nuevo análisis

1 Carga2 Perfil base3 Diagnóstico4 Reporte diagnóstico5 Exportación

RAMA OPCIONAL: REMEDIACIÓN

La reauditoría te permite comparar los hallazgos antes/después si decides limpiar una copia del dataset. Puedes volver al informe diagnóstico o a la Exportación en cualquier momento.

Volver al reporte diagnóstico

Aplicar y verificar

Descargá el bundle y el CSV fuente, ejecutá el script de remediación en tu entorno Python local y subí el resultado junto con el recibo.

Ejecución Python verificada.

PYTHON3.14.6

PANDAS3.0.3

PLATAFORMA macOS-26.5.2-arm64-arm-64bit-Mach-0

DURACIÓN696 ms

FILAS10

COLUMNAS12

HASH SALIDA e02915ce...6609a6

HASH RECIBO 709737ab...eca002

Remediación reauditada.

Comparación reproducible con el mismo motor determinista (runAudit).

SCORE ANTES75

SCORE DESPUÉS75

HALLAZGOS ANTES3

HALLAZGOS DESPUÉS3

REGLAS CORREGIDAS0

REGLAS PERSISTENTES2

REGLAS NUEVAS0

Persistentes: rule:null-values, rule:trim-whitespace

Ir a Exportación

AURA

AyudaHistorial

CASABERO · AURA · 2026

Fuente: evidencia E2E de AURA. La rama de remediación pertenece al pipeline normal.

5.5. Fragmentos de código explicados

Los fragmentos siguientes no pretenden reemplazar la lectura completa del repositorio, sino mostrar de forma breve cómo se implementan los principios técnicos de AURA. Cada fragmento se acompaña de una explicación y de su límite.

5.5.1. Lectura local del CSV

El módulo de lectura local convierte el archivo en filas y columnas dentro del navegador. La función detecta campos y delimitador, y entrega el contenido al motor de auditoría.

```
const { data, meta } = await parseCsv(uploadedFile);  
// data: filas del CSV como objetos  
// meta.fields: nombres de columna  
// meta.delimiter: separador detectado
```

Este fragmento demuestra procesamiento local sin backend. No demuestra rendimiento con archivos de millones de filas.

5.5.2. Regla determinista representativa

Una regla determinista inspecciona valores de texto para detectar espacios invisibles al inicio o final. El hallazgo resultante incluye severidad, columna y conteo.

```
if (typeof value === 'string' && value !== value.trim()) {  
  issues.push({  
    ruleName: 'Espacios Fantasma (Trim)',  
    severity: 'info',  
    column: field,  
    count: 1,  
    affectedPercentage: ...  
  });  
}
```

Este fragmento demuestra reglas reproducibles. No demuestra que una sola regla cubra todos los casos posibles de limpieza textual.

5.5.3. Construcción del paquete de evidencia

Antes de la interpretación asistida, AURA construye un paquete con fingerprint, tiempos, columnas procesadas y resultado del informe técnico. Este paquete reduce la necesidad de exponer el dataset completo.

```
const datasetFingerprint = fingerprintDataset(data, meta.fields);
const evidence = buildAuditEvidence({
  fileName, datasetFingerprint,
  startedAt, completedAt,
  rowsProcessed, columnsProcessed,
  report: auditResult,
  trace: trace.events
});
```

Este fragmento demuestra trazabilidad del procesamiento. No demuestra resistencia absoluta frente a manipulación externa.

5.5.4. Generación de script desde acciones aprobadas

El generador controlado produce código únicamente para acciones aprobadas. Las acciones rechazadas o pendientes no generan transformaciones.

```
function renderActionV2(action, columnRef, columnRegistry): string {
  switch (action.actionType) {
    case 'trim_whitespace':
      return `df_clean[${writeTarget}] = df_clean[${writeTarget}].astype(str).str.strip()`;
    // otros tipos de acción
  }
}
```

Este fragmento demuestra que el código deriva de decisiones humanas y reglas internas, no de texto libre del LLM. No demuestra que el script generado sea óptimo para todos los dominios.

5.5.5. Revisión humana y bloqueo

Antes de aprobar el contrato de script, el sistema realiza una verificación fresca. Si hay inconsistencia, por ejemplo un hash alterado, la aprobación se bloquea.

```
const freshVerification = verifyScriptContractV2(
  scriptContractV2, remediationPlanV2, contextResult.buildContext
);
if (!freshVerification.valid) {
  setStage('pending');
  return; // bloquea la aprobación
}
```

Este fragmento demuestra una política de fallo cerrado ante inconsistencias. No demuestra que todas las formas posibles de manipulación sean detectadas.

5.5.6. Bloqueo de acciones no aprobadas

El renderer rechaza cualquier acción cuyo estado no sea approved. La comprobación ocurre antes de resolver columnas o generar texto Python, por lo que una acción pendiente o rechazada no puede filtrarse hacia el script.

```
if (action.approvalStatus !== 'approved') {
  throw new ScriptRendererError(
    'RENDER_ACTION_NOT_APPROVED',
    `Action ${action.actionId} is not approved`,
  );
}
```

Este control transforma la decisión humana en una precondition técnica. La interfaz no solo registra la aprobación: el renderer exige ese estado para producir código.

5.5.7. Registro canónico de columnas como autoridad

Las referencias de columna recibidas por el renderer se consideran no confiables. AURA busca el columnId en un registro canónico y compara nombre, posición, ordinal de duplicado, literal Python y estados de ambigüedad. Una diferencia bloquea la acción.

```
const registryCol = registry.byColumnId.get(columnRef.columnId);
if (!registryCol) {
  throw new ScriptRendererError('RENDER_COLUMN_NOT_IN_REGISTRY');
}
if (!columnsMatch(columnRef, registryCol) || registryCol.isAmbiguous) {
  throw new ScriptRendererError('RENDER_COLUMN_MISMATCH');
}
```

Este mecanismo evita que una respuesta del modelo o una manipulación externa cambie el destino de la transformación hacia otra columna con nombre parecido.

5.5.8. Plantillas deterministas y trazabilidad dentro del script

El renderer utiliza un switch cerrado sobre tipos de acción. Por ejemplo, la normalización de espacios siempre produce la misma plantilla para los mismos parámetros. El encabezado del script puede incorporar el hash del prompt, el hash de entrada, el recibo del diagnóstico y la referencia al paquete de evidencia.

```
switch (action.actionType) {
  case 'trim_whitespace':
    return renderTrimWhitespace(action, resolvedCol);
  case 'normalize_placeholders':
    return renderNormalizePlaceholders(action, resolvedCol);
  case 'convert_disguised_numbers':
    return renderConvertDisguisedNumbers(action, resolvedCol);
  default:
```

```
    throw new ScriptRendererError('RENDER_UNSUPPORTED_ACTION');
  }
```

El mismo conjunto de acciones, columnas y parámetros genera el mismo texto. Esta propiedad facilita revisión, pruebas unitarias, comparación de hashes y reproducción del artefacto.

5.5.9. Bundle y recibo de ejecución con hashes

El bundle se firma calculando SHA-256 sobre una serialización canónica que excluye el propio bundleHash. El recibo se valida contra el runId, el hash del script aprobado, el hash del texto, el dataset fuente y el CSV resultante.

```
export const computePythonBundleHash = (bundle) =>
  sha256hex(canonicalJson(serializeBundleForHash(bundle)));

if (hashArtifact(afterCsv) !== receipt.afterDatasetSha256) {
  errors.push('output CSV hash mismatch');
}
```

La validación también rechaza estados imposibles, como una ejecución aprobada con sintaxis fallida, un éxito sin hash de salida o un fallo que pretende certificar un CSV. Esta política fail-closed impide convertir un error en resultado positivo.

5.5.10. Reauditoría del CSV importado

Después de validar el recibo, AURA ejecuta runReaudit sobre el CSV anterior y posterior. La evidencia resultante se combina con HealthDelta y se persiste en la corrida.

```
const reaudit = runReaudit(
  beforeCsv,
  afterCsv,
  run.input.evidenceEnvelopeRef,
  { delimiter: ';' },
);
const healthDelta = calculateHealthDelta(
  reaudit.beforeReport,
  reaudit.afterReport,
);
```

El diseño evita utilizar el LLM para decidir si el dataset mejoró. La comparación antes/después vuelve a depender del motor determinista, de la identidad exacta de los archivos y de métricas calculadas por código reproducible.

5.6. Contribuciones técnicas e innovaciones de AURA

La innovación de AURA no se reduce a integrar un modelo de lenguaje en una interfaz de calidad del dato. Su contribución está en la distribución de autoridad y en la cadena verificable que conecta evidencia, interpretación, decisión, código, ejecución y reauditoría.

Contribución	Riesgo convencional	Respuesta de AURA	Evidencia verificable
Diagnóstico restringido	El modelo interpreta el dataset sin límites	EvidenceEnvelopeV2 y contrato de respuesta	Prompt/input hash, referencias y validación
Código determinista	El LLM genera Python libre y variable	Vocabulario cerrado + renderer de plantillas	ScriptContractV2 y scriptHash
HITL real	Aprobación meramente visual	El renderer exige acciones aprobadas	accepted/rejected action IDs
Ejecución aislada	La aplicación modifica el archivo original	Runner externo sobre una copia	Bundle, receipt y corrected.csv
Integridad criptográfica	Artefactos no vinculados entre sí	SHA-256 para entrada, script, bundle y salida	receiptHash, bundleHash y dataset hashes
Reauditoría reproducible	Mejora inferida por narrativa	Mismo motor antes y después	Reglas resueltas, persistentes y nuevas
Comparación multimodelo	Ganador por intuición o una sola corrida	Matriz 3×3×3 y recomendaciones por finalidad	Campaña exportable y oráculo congelado

Tabla 9. Síntesis de contribuciones técnicas y mecanismos de control.

5.7. Estrategia de pruebas y aseguramiento de calidad

La estrategia de verificación sigue una pirámide adaptada al riesgo. Las funciones puras — hashes, canonicalización, guards, evaluadores y agregadores— se cubren con pruebas unitarias. Los componentes se prueban con Testing Library para validar estados visibles. Los recorridos que dependen de navegación, descargas o importación de archivos se ejecutan con Playwright. TypeScript typecheck y el build de producción actúan como gates adicionales, pero no sustituyen la prueba de uso real.

El recorrido E2E de remediación prepara el bundle, ejecuta el runner real, valida receipt.json, importa corrected.csv, reaudita y abre la exportación. Un caso negativo altera el recibo y confirma que AURA lo rechaza. El paquete ZIP se inspecciona para comprobar que incluye script, bundle, recibo, CSV corregido y resultados antes/después, pero no source.csv. Esta prueba cubre la promesa funcional completa, no solo la presencia de botones.

El Laboratorio requiere otra clase de evidencia. Los tests validan que utiliza el mismo constructor de entrada, prompt, esquema y parser que el pipeline normal; que conserva respuestas fallidas; que toma el modelo observado de Ollama; y que los resultados se agregan sin inventar métricas. La paridad evita que la campaña evalúe un flujo distinto del producto.

La verificación humana realizada durante el cierre detectó problemas que los tests no habían anticipado: texto desbordado, avance invisible, nombres obsoletos de modelos, límites de salida insuficientes y dependencia indebida de HITL en el Laboratorio. Esos incidentes se

documentan en la campaña piloto y motivaron cambios de UI, parámetros y contrato. La lección metodológica es que un flujo crítico necesita validación automática y recorrido humano.

Tabla 2. Capas de prueba y evidencia aportada

Capa	Ejemplo	Qué demuestra	Qué no demuestra
Unidad	Guards, hashes, scoring	Lógica reproducible	Recorrido de usuario
Componente	Estados y mensajes	Contrato visible	Integración completa
E2E	Runner, ZIP, importación	Flujo ejecutable	Generalización a todo dataset
Campaña	27 diagnósticos	Comparación contextual	Superioridad universal
Recorrido humano	Uso publicado	Claridad práctica	Usabilidad estadística

Elaboración propia.

5.8. Decisiones de arquitectura y compromisos de diseño

La arquitectura final no surgió de elegir una tecnología y adaptar después el problema. Se construyó a partir de restricciones verificables. La primera fue evitar que el archivo completo saliera del navegador durante el perfilamiento. La segunda fue impedir que una respuesta probabilística se tratara como una instrucción ejecutable. La tercera fue conservar suficiente evidencia para reconstruir una corrida sin almacenar innecesariamente el CSV original. Estas restricciones explican la separación entre motor determinista, envelope de evidencia, diagnóstico LLM, plan de remediación, renderer, runner y reauditoría. Cada componente reduce el conjunto de decisiones que puede tomar el siguiente y deja un artefacto observable para el control posterior.

El enfoque local-first aporta privacidad operativa y autonomía, pero introduce compromisos. El navegador limita memoria, tiempo de cómputo y acceso directo a Python; Ollama depende del hardware de cada equipo; y la persistencia local no ofrece las mismas garantías que una plataforma centralizada. AURA asume esos límites y los hace visibles. El perfilamiento se ejecuta en cliente; el diagnóstico consulta un servicio local explícitamente configurado; la ejecución Python se prepara como un bundle y ocurre fuera del navegador; y la evidencia pesada no se restaura silenciosamente después de recargar la sesión. La pérdida de comodidad es deliberada cuando evita afirmar que una ejecución ocurrió sin disponer de sus archivos y recibos.

La decisión de usar contratos versionados responde a un problema práctico: una interfaz puede seguir mostrando resultados aunque el significado interno de un campo haya

cambiado. Por ello, snapshot, respuesta diagnóstica, recibo, plan y bundle incorporan identificadores y versiones. El validador no comprueba únicamente que exista un JSON; verifica cobertura, referencias, correspondencia entre modelo solicitado y observado y hashes derivados de una serialización canónica. Cuando la política de revisión humana debe prevalecer sobre una salida del modelo, AURA conserva la respuesta cruda, registra el incumplimiento y produce una vista efectiva gobernada. El Laboratorio puntúa la salida cruda para no regalar cumplimiento artificial, mientras el producto aplica la protección necesaria para no degradar seguridad.

Otro compromiso afecta al script. Permitir código libre generado por un LLM ampliaría el repertorio de transformaciones, pero también el espacio de comportamiento no probado. AURA adopta la opción contraria: el plan utiliza tipos de acción cerrados y el renderer conoce cómo traducir cada tipo a Python/Pandas. Esta elección no elimina todo riesgo, porque una regla autorizada todavía puede ser inapropiada para un dominio, pero hace posible revisar la acción, probar la plantilla, calcular el hash del script y vincular la ejecución con el contrato aprobado. El sistema sacrifica flexibilidad abierta a cambio de explicabilidad y control.

Tabla 3. Relación entre restricciones, decisiones y consecuencias

Restricción	Decisión aplicada	Consecuencia verificable
Privacidad del CSV	Perfilamiento local y payload reducido	El original no se incluye en el ZIP diagnóstico
LLM probabilístico	JSON V2 y validador estricto	Las salidas inválidas se conservan como fallos
Autoridad humana	HITL antes de renderizar	Solo las acciones aprobadas entran en el script
Python fuera del navegador	Bundle + runner + receipt	La ejecución se demuestra con hashes y entorno
Comparación reproducible	Protocolo, warm-up y modelo observado	Cada celda conserva denominadores y fallos

Elaboración propia a partir de la arquitectura implementada.

5.9. Ciclo de vida del dato y fronteras de confianza

El ciclo comienza cuando la persona selecciona un CSV. El archivo se representa como un objeto local del navegador y se procesa para inferir delimitador, columnas, tipos, estadísticas y activaciones de reglas. En esta fase, los nombres de columnas y los valores son datos no confiables: pueden contener texto que parezca una instrucción, secuencias de escape o contenido sensible. AURA no los interpreta como órdenes. El motor produce identificadores

internos y un registro de hallazgos; después el constructor de evidencia decide qué fragmentos son visibles para el modelo según el método de entrada. La minimización se aplica antes de la inferencia, no como una corrección posterior.

La frontera entre AURA y Ollama se trata como una invocación externa aunque ambos se ejecuten en el mismo equipo. El request especifica modelo y parámetros; la respuesta del servidor aporta el modelo observado y métricas de inferencia. Esta distinción evita certificar circularmente el modelo solicitado. Si el servidor no declara el modelo, el recibo no inventa uno. Si declara uno diferente, la corrida no puede presentarse como válida. De forma análoga, el hash de la respuesta se calcula sobre la salida recibida, antes de normalizaciones de gobernanza, y permite demostrar que la vista efectiva procede de una respuesta cruda concreta.

La remediación introduce una frontera distinta. La persona pasa de lectora a autoridad de decisión: puede aprobar o rechazar cada acción. La interfaz muestra tipo de acción, regla, columna, alcance y número de evidencias; cero evidencias no significa ausencia de hallazgo, sino que la regla puede basarse en una medida agregada sin muestra exportable. Tras la aprobación, el renderer produce un script inmutable para ese contrato. El bundle incluye referencias al dataset y al script, pero el runner recibe el archivo por separado. Esta separación permite excluir el original del expediente final y, al mismo tiempo, comprobar que el archivo ejecutado coincide con el hash esperado.

El CSV corregido se considera un nuevo artefacto, no una sobrescritura del original. La reauditoría aplica el mismo motor determinista y compara reglas antes y después. La comparación informa corregidas, persistentes y nuevas, pero no atribuye automáticamente causalidad semántica: una reducción puede proceder de una acción aprobada y aun así requerir validación de negocio. El expediente incluye el CSV corregido solo cuando existe una ejecución verificada; advierte que puede conservar datos personales. Esta regla evita que la exportación diagnóstica prometa una corrección inexistente y obliga a diferenciar análisis, propuesta, ejecución y verificación.

Tabla 4. Fronteras de confianza del flujo AURA

Frontera	Entrada no confiable	Control	Evidencia
CSV → motor	Nombres y valores	Parser, tipos y reglas	Perfil y hallazgos
Motor → LLM	Payload con contenido del dataset	Envelope, secciones y prompt de sistema	Snapshot y hashes

Frontera	Entrada no confiable	Control	Evidencia
LLM → producto	JSON probabilístico	Schema y referencias	Respuesta cruda y recibo
Persona → renderer	Decisión contextual	Aprobación por acción	Plan y contrato
Bundle → runner	Archivos locales	Hash, py_compile y recibo	Corrected.csv y receipt.json
Resultado → exportación	CSV potencialmente sensible	Preflight y manifiesto	ZIP con privacidad declarada

Elaboración propia.

5.10. Estados, invalidación y recuperación del flujo

Un pipeline gobernado necesita controlar no solo los resultados, sino su vigencia. Si la persona consulta una etapa anterior, la evidencia no cambia. Si modifica el diagnóstico o selecciona otra configuración, los artefactos derivados dejan de ser válidos. AURA implementa invalidación descendente: cambiar una decisión que alimenta el plan invalida script, aprobación, bundle, receipt y reauditoría; cambiar el script invalida aprobación y ejecución; destruir la sesión elimina datos locales. Esta política evita reutilizar un recibo correcto para un script que ya no coincide con él.

La recuperación de sesión conserva metadatos livianos, pero excluye bytes del CSV corregido y evidencia de reauditoría que no puede serializarse de manera responsable. Después de recargar, la persona debe reimportar corrected.csv y receipt.json. La decisión puede parecer restrictiva, aunque protege dos propiedades: no acumular datos personales en localStorage y no reconstruir una ejecución a partir de indicadores incompletos. La interfaz debe explicar esa situación con lenguaje operativo y ofrecer el siguiente paso, porque una política segura que no se entiende se percibe como un fallo.

Los estados de fallo también forman parte del producto. Una respuesta truncada, un JSON inválido o una referencia no soportada no se descartan; generan recibo inválido, código de error y hash de respuesta cruda cuando existe. La campaña formal conserva esos fallos en el denominador. En el pipeline normal, la persona puede reintentar o continuar sin diagnóstico, pero la exportación no presenta un recibo inválido como certificación. La diferencia entre permitir avanzar y declarar validez resulta central: AURA favorece continuidad operativa sin borrar el límite de evidencia.

5.11. Estrategia de pruebas por capas

La verificación se organiza por riesgo. Las funciones puras —hashes, normalización, validadores, cálculo de métricas y construcción de planes— se prueban con tests unitarios y

fixtures pequeños. Los componentes de interfaz se prueban con Testing Library cuando el comportamiento puede verificarse sin navegador completo: mensajes, estados, habilitación de botones y exclusión mutua entre éxito y fallo. Las integraciones de exportación comprueban estructura, privacidad y preflight. Playwright se reserva para recorridos que dependen de descargas, navegación entre etapas, ejecución del runner y reimportación de archivos.

Los gates de typecheck y build tienen funciones diferentes. TypeScript detecta incompatibilidades entre contratos antes de ejecutar; el build confirma que la aplicación de producción puede generarse con las dependencias y rutas reales. Ninguno sustituye a los tests. Un error significativo del desarrollo ilustra esta separación: el código compilaba y las suites focalizadas pasaban, pero la primera campaña reveló que el runner del Laboratorio no compartía exactamente el pipeline del diagnóstico normal. Solo una ejecución extremo a extremo mostró que la evidencia experimental no era comparable con el producto. La corrección consistió en unificar la fábrica de entrada, el parser, el recibo y la captura del modelo observado.

El recorrido E2E de remediación reproduce la acción humana completa: cargar un dataset, obtener diagnóstico, seleccionar acciones, generar contrato, revisar script, preparar bundle, ejecutar el runner real, importar corrected.csv y receipt.json, reauditar y exportar el ZIP. Los tests negativos alteran el recibo o fuerzan desbordamientos para comprobar rechazo y estabilidad visual. El valor de este recorrido no es afirmar ausencia de defectos, sino demostrar que las fronteras críticas funcionan juntas y que los artefactos descargados pueden cerrar el ciclo.

Tabla 5. Capas de verificación y propósito

Capa	Qué demuestra	Qué no demuestra
Unitarias	Lógica determinista y casos límite	Recorrido humano completo
Componentes	Estados y mensajes de la UI	Descargas y servicios reales
Integración	Contratos entre módulos y exportación	Usabilidad externa
Typecheck	Coherencia estática	Corrección de negocio
Build	Empaquetado de producción	Resultado experimental
Playwright	Flujo de navegador y archivos	Generalización a todos los equipos
Campaña	Comportamiento repetido en una matriz	Validez universal

Elaboración propia.

5.12. Auditoría de la evidencia exportada

El paquete de evidencia se diseñó como expediente, no como una carpeta de descargas sin relación. `manifest.json` enumera cada artefacto con tamaño y SHA-256. El JSON técnico conserva perfil, hallazgos, diagnóstico estructurado, snapshot, recibo y estado de validación. El CSV de hallazgos ofrece una vista tabular, pero no reemplaza al JSON porque omite el recibo. El PDF interpreta los resultados para lectura humana. Cuando existe remediación verificada, el ZIP añade `script aprobado`, `bundle`, `receipt`, `corrected.csv`, resultado de reauditoría y resumen antes/después. El dataset original permanece fuera del paquete.

La evidencia de campaña tiene otra frontera. Su propósito es comparar diagnósticos; por ello no incluye HITL ni generación de script como componentes de la puntuación. `campaign.json` congela protocolo y unidades; `runs.csv` conserva una fila por corrida; `results-summary.json` agrega dimensiones; `selected-configuration.json` traduce una recomendación al pipeline; `methodology.md` y `glossary.md` explican reglas y términos. Un paquete parcial no debe llamarse reporte formal: el guard exige campaña terminada, todas las unidades intentadas y representantes válidos. Los fallos permanecen visibles en `runs.csv` y en los denominadores.

La auditabilidad depende tanto de presencia como de ausencia. El expediente diagnóstico debe contener el prompt exacto, pero no API keys; el ZIP de remediación debe contener `corrected.csv` cuando fue verificado, pero no `source.csv`; el reporte de reauditoría debe conservar métricas, pero no duplicar `rawCsv`. Estas propiedades se prueban explícitamente. Verificar que un secreto o un dataset no está presente es tan importante como verificar que un recibo sí lo está, porque la exportación es la superficie donde la evidencia técnica puede convertirse en una fuga de información.

6. Código fuente, datos y reproducibilidad

6.1. Código fuente

AURA se desarrolla en un repositorio versionado con frontend React/TypeScript, contratos LLM, motor de auditoría, servicios de benchmark, generación de reportes, exportación y runners de remediación. La memoria no utiliza el número de commits como medida de calidad. La evidencia relevante es la correspondencia entre requisito, implementación, prueba y artefacto exportado.

El historial incluye identidades de agentes automatizados empleados por el autor para organizar y revisar código. Estas identidades no representan coautores humanos ni transferencia de propiedad; reflejan herramientas operadas bajo la dirección del autor. La responsabilidad de aceptar cambios, ejecutar pruebas e interpretar resultados corresponde al autor.

La versión de aplicación capturada en la campaña 2 fue el commit `f1efed651943ac92d027a0d52ddc0ce25344c4d1`. El snapshot se conserva por corrida y permite vincular los resultados con el estado del software. La memoria no depende de que el repositorio permanezca sin cambios después del experimento.

6.2. Datos analizados

El desarrollo utilizó casos exploratorios y datasets controlados. Titanic permitió ejercitar carga, tipos, nulos y duplicados en un conjunto ampliamente conocido, pero no se utiliza como ground truth exhaustivo. `controlled_customers_phase8.csv` permitió probar 50 filas, 15 columnas y 29 hallazgos esperados durante fases de desarrollo. `synthetic_ground_truth.csv`, con 15 filas y nueve columnas, se seleccionó para las campañas porque dispone de un registro canónico y de muestras diseñadas para reglas concretas.

Un dataset controlado no pretende representar una distribución empresarial. Su ventaja es epistemológica: permite saber qué se espera y verificar el contrato. Su limitación es externa: los resultados no garantizan comportamiento sobre archivos más grandes, otros idiomas, columnas ambiguas o reglas de negocio. Por ello, AURA exige que una campaña futura se

cree a partir de un dataset y un ground truth compatibles; la automatización de ese orquestador queda como mejora futura.

El hash SHA-256 de `synthetic_ground_truth.csv` en la campaña definitiva fue `4e7d358f2141c6463146417a66f1c2312c7c3cdf6a39005780c92b061ff7ac49`. El ground truth y el esquema poseen hashes separados. Esta separación impide sustituir el CSV, la anotación o la estructura sin cambiar el snapshot experimental.

6.3. Ground truth y evidencia canónica

El ground truth se representa como un conjunto de claves canónicas `ruleId + columnName + scope`. La comparación exacta evita valorar como equivalente un hallazgo referido a otra columna o ámbito. Los `issueId` y `evidenceRef` permiten rastrear la explicación, pero la métrica primaria usa la identidad estable de la regla, la columna y el scope.

El registro canónico funciona como oráculo de cobertura para el diagnóstico. Por esa razón, el F1 del Laboratorio es descriptivo y no pondera el índice equilibrado. En cambio, la evaluación del motor determinista puede utilizar TP, FP y FN cuando la anotación incluye negativos o una política explícita para detecciones no anotadas. AURA evita convertir automáticamente toda detección adicional en falso positivo si el dataset no tiene etiquetas negativas exhaustivas.

Cada respuesta conserva `evidenceRefs` visibles. Para valores sensibles, el envelope puede utilizar referencias SHA-256 o muestras redactadas. El validador acepta una abstracción hash únicamente cuando la evidencia de la incidencia contiene un SHA-256 real de 64 caracteres. Esta regla permite hablar de una muestra protegida sin inventar el valor original.

6.4. Verificación del software y evidencia de extremo a extremo

El cierre técnico incorporó pruebas unitarias, typecheck, build y recorridos E2E con runner y ZIP reales. La evidencia no se limita a afirmar que la interfaz permite descargar archivos; los tests validan recibos, hashes, ausencia del CSV original en el paquete exportado, rechazo de manipulación y conservación de errores.

Durante el cierre técnico se completó un recorrido humano publicado con `synthetic_ground_truth.csv`, Qwen3.5 4B y el método Evidencia completa. El diagnóstico conservó 15 hallazgos; la persona aprobó cuatro acciones y rechazó once; el runner validó

sintaxis y ejecutó el script sobre una copia; el resultado pasó de 15 a 14 filas y mantuvo nueve columnas; la reauditoría pasó de 15 a 11 hallazgos, con cuatro hallazgos corregidos y ninguna regla nueva. El ZIP final contenía 28 archivos y sus 27 entradas declaradas coincidían en hash y tamaño.

Este recorrido no demuestra que cualquier dataset mejore ni que toda decisión humana sea correcta. Demuestra que la cadena técnica completa puede ejecutarse, verificarse y volver a auditarse bajo un caso controlado.

6.5. Expediente reproducible y privacidad

La exportación del pipeline normal reúne PDF, JSON técnico, CSV de hallazgos, prompt, respuesta, recibo, plan, contrato, script y, cuando existe una ejecución verificada, bundle, receipt, corrected.csv y comparación antes/después. manifest.json registra ruta, tamaño y hash de cada artefacto. El CSV original no se incluye; queda identificado por su SHA-256.

El Laboratorio exporta un expediente distinto porque su objeto es comparar diagnósticos. Incluye campaign.json, runs.csv, results-summary.json, report.md, report.pdf, methodology.md, glossary.md, selected-configuration.json y manifest.json. No incluye scripts ni HITL, ya que esos elementos pertenecen al pipeline normal. Los fallos permanecen en campaign.json y runs.csv para medir estabilidad.

La evidencia sensible no se persiste automáticamente entre sesiones. AURA conserva metadatos necesarios, pero los bytes del CSV corregido deben volver a importarse después de una recarga. Esta decisión introduce fricción deliberada para reducir permanencia de datos personales en el navegador.

6.6. Entorno congelado de la campaña definitiva

La campaña 2 se ejecutó en Windows con 24 núcleos lógicos y 32 GiB de memoria capturados por el navegador. Ollama 0.31.1 atendió las llamadas locales. Los modelos se ejecutaron con cuantización UD-Q4_K_XL y digests locales verificados. Los parámetros comunes fueron temperature 0,1, top_p 0,9, num_ctx 32.768, num_predict 8.192, think false, keep_alive 10 minutos y timeout 900 segundos.

El hardware condiciona latencia, pero no se incorpora al índice como una constante universal. La eficiencia se normaliza dentro de la campaña. Cambiar GPU, CPU, memoria,

runtime o cuantización requiere una campaña nueva; copiar selected-configuration.json a otro equipo no garantiza reproducir la misma latencia ni la misma tasa de cumplimiento.

Tabla 6. Artefactos del expediente de Laboratorio

Artefacto	Contenido	Uso
campaign.json	Campaña, corridas, recibos y evaluación	Auditoría técnica
runs.csv	Una fila por corrida	Análisis tabular
results-summary.json	Matriz, scores y recomendaciones	Fuente de gráficos
report.md / report.pdf	Informe humano reproducible	Lectura y defensa
methodology.md	Oráculo y ponderaciones	Interpretación
selected-configuration.json	Modelo, entrada y parámetros	Aplicación al pipeline
glossary.md	Definiciones simples	Comprensión
manifest.json	Hashes y tamaños	Integridad

Fuente: exportación oficial de la campaña 2.

7. Diseño experimental del Laboratorio

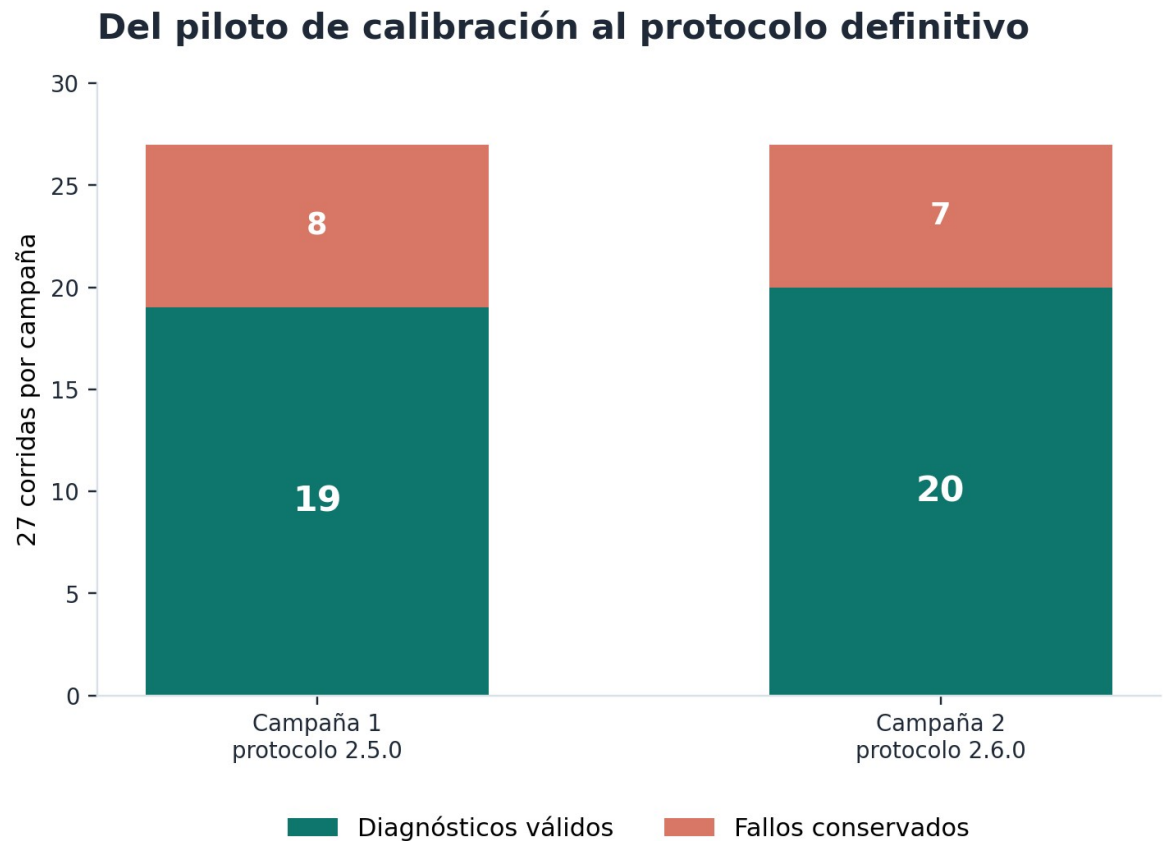
El Laboratorio de AURA compara modelos locales y métodos de entrada utilizando el mismo pipeline diagnóstico que la auditoría normal. Su función no es generar scripts ni sustituir la revisión humana, sino consolidar respuestas, medirlas contra un ground truth y traducir el resultado en una configuración de uso. El diseño definitivo se construyó después de un piloto que expuso diferencias entre producto y experimento, dependencias indebidas de HITL, límites de salida insuficientes y una puntuación con doble conteo.

7.1. Campaña 1: piloto de calibración

La primera campaña se ejecutó con el protocolo 2.5.0 sobre `synthetic_ground_truth.csv`. Intentó las 27 unidades y registró 19 diagnósticos válidos y ocho fallos. El experimento fue útil porque demostró que los modelos y métodos podían recorrer la matriz, pero también reveló problemas conceptuales y de producto. El Laboratorio conservaba estados de revisión humana y script que no pertenecían a la evaluación diagnóstica; la interfaz no distinguía con claridad corridas válidas y fallidas; y el índice equilibrado otorgaba peso simultáneo a exactitud y contrato aun cuando la cobertura ya era una obligación del esquema.

Durante las primeras ejecuciones también se observaron respuestas truncadas por `num_predict=1.600`, diferencias de parámetros entre el pipeline principal y el runner formal, validaciones de `requiresHumanReview` que el modelo no podía inferir con todos los métodos de entrada y ausencia de un paquete descargable para fallos. Esos incidentes no se ocultan: forman parte de la evidencia de maduración. La campaña 1 se conserva en `experiments/tests/test_campaña` y no se mezcla estadísticamente con la campaña 2.

Figura 7. Evolución del piloto al protocolo definitivo



Fuente: elaboración propia a partir de las exportaciones de las campañas 1 y 2.

Tabla 7. Problemas detectados en el piloto y ajustes aplicados

Problema	Riesgo	Ajuste antes de campaña 2
HITL y script en Laboratorio	Mezclar diagnóstico y remediación	Laboratorio restringido al diagnóstico
Pipeline y runner distintos	Violación del principio experimental	Constructor, prompt, parser y validador comunes
Salida máxima insuficiente	JSON truncado	num_predict 8.192 y num_ctx 32.768
Puntuación con doble premio	Índice inflado	GT y contrato como control/gate, sin peso
Progreso y fallos poco visibles	Operación opaca	Stream real, estados, colores y pausa segura
Sin evidencia descargable de fallo	Diagnóstico difícil	Paquete de evidencia para corridas fallidas
Catálogo estático de modelos	Modelo solicitado obsoleto	Lista dinámica desde Ollama

Elaboración propia a partir del registro de incidencias y del protocolo 2.6.0.

7.2. Campaña 2: protocolo definitivo

La segunda campaña utilizó el protocolo `aura.oe4.final-evaluation.v2` versión 2.6.0. El dataset, su esquema, el ground truth, los modelos, los métodos de entrada, los parámetros y las ponderaciones se congelaron antes de iniciar. La campaña se identificó como `campaign:oe4:v2:20260715011720492` y terminó con validez formal. Cada corrida conserva

el commit de la aplicación, hardware, runtime, modelo, digest, inferencia, prompt, entrada, respuesta y recibo.

Tabla 8. Diseño congelado de la campaña 2

Elemento	Valor
Dataset	synthetic_ground_truth.csv; 15 filas; 9 columnas
SHA-256	4e7d358f2141c6463146417a66f1c2312c7c3cdf6a39005780c92b061ff7ac49
Modelos	Qwen3.5 4B; Gemma 4 E4B; Smollm3 3B
Entradas	Contexto mínimo; Evidencia equilibrada; Evidencia completa
Repeticiones	3 por combinación
Diagnósticos evaluados	27
Calentamientos excluidos	9
Llamadas reales	36
Inferencia	temperature 0,1; top_p 0,9; num_ctx 32.768; num_predict 8.192
Entorno	Windows; 24 núcleos lógicos; 32 GiB; Ollama 0.31.1

Fuente: *campaign.json* y *selected-configuration.json* de la campaña 2.

7.3. Métodos de entrada

Contexto mínimo incluye resumen del dataset, esquema y registro mínimo de hallazgos. No incorpora muestras observadas. Su finalidad es comprobar si el modelo puede producir la estructura requerida con la menor exposición y longitud. Evidencia equilibrada añade estadísticas de columnas, reglas activadas y muestras limitadas. Evidencia completa agrega políticas, manifiestos, registro de columnas y anclajes exactos. Los nombres internos `prompt_libre`, `smart_sample` y `recommended` se conservan en los artefactos por compatibilidad histórica; la interfaz y esta memoria utilizan los tres nombres definitivos.

Los métodos no son prompts libres en sentido cotidiano. Los tres comparten instrucción del sistema, esquema de respuesta, envelope y validación. Lo que cambia es el payload visible. El recibo certifica método solicitado, método efectivo, secciones, `promptHash`, `inputHash` y `responseSchemaHash`. Esta evidencia permite comprobar que una celda se ejecutó con la entrada que declara.

Más contexto no implica automáticamente más calidad. Puede aportar referencias útiles, pero también aumenta longitud y superficie narrativa. La campaña definitiva encontró que Contexto mínimo fue suficiente para las combinaciones con mayor puntuación, mientras que los métodos más extensos conservaron cobertura pero generaron claims no soportados o fallos en el modelo más pequeño.

7.4. Métricas automáticas

La alineación con ground truth utiliza coincidencia exacta de ruleId + columnId + scope. Precisión es $TP/(TP+FP)$, recall es $TP/(TP+FN)$ y F1 es $2 \cdot \text{precisión} \cdot \text{recall} / (\text{precisión} + \text{recall})$ (Sokolova & Lapalme, 2009). En este experimento, esas métricas controlan cobertura de un registro conocido; no miden creatividad, estilo ni descubrimiento abierto.

Fiabilidad es la proporción de corridas válidas entre intentadas. Cumplimiento contractual comprueba JSON, cobertura, referencias, modelo observado y reglas de gobernanza. Soporte de evidencia mide si las referencias declaradas pertenecen al envelope visible. Seguridad frente a claims sin soporte penaliza columnas inventadas, valores citados sin evidencia y afirmaciones incompatibles con el payload. Eficiencia normaliza latencia y tokens dentro de la campaña.

El índice equilibrado usa fiabilidad 35 %, soporte de evidencia 25 %, seguridad frente a claims sin soporte 20 % y eficiencia 20 %. Alineación con GT recibe 0 % y contrato 0 %: la primera es descriptiva y el segundo es gate. Esta ponderación es una preferencia explícita de ayuda a la decisión, no una verdad matemática. Por ello, el informe conserva todas las dimensiones y recomendaciones por finalidad.

Tabla 9. Definiciones operativas de las métricas

Métrica	Cálculo	Interpretación	Riesgo de lectura
Precisión	$TP/(TP+FP)$	Proporción de hallazgos declarados correctos	Requiere negativos anotados
Recall	$TP/(TP+FN)$	Cobertura de hallazgos esperados	No mide utilidad narrativa
F1	Media armónica	Equilibrio precisión-recall	Contrato ya exige cobertura
Fiabilidad	Válidas/intentas	Estabilidad operativa	Solo tres repeticiones
Evidencia	Refs válidas/esperadas	Anclaje al payload	No prueba verdad de dominio
Claims soportados	Ausencia de claims inválidos	Honestidad respecto de la entrada	Validador cerrado
Eficiencia	Normalización de latencia y tokens	Coste relativo	Depende del hardware

Fuente: methodology.md y glossary.md de la campaña 2.

7.5. Validez y amenazas previstas

La validez interna mejora al congelar prompt, esquema, parámetros y orden, excluir warm-ups y conservar fallos. No obstante, tres repeticiones no permiten estimar con precisión una distribución completa. La validez de constructo está limitada porque el oráculo mide

cobertura exacta, no toda la utilidad de una explicación. La validez externa es reducida: un único dataset de 15 filas no representa todos los dominios ni tamaños.

La validez tecnológica depende de una cuantización y un runtime concretos. La cuantización reduce memoria y hace viable la inferencia local, pero puede cambiar el comportamiento frente al modelo de mayor precisión (Frantar et al., 2022; Lin et al., 2024). La validez humana no se evalúa estadísticamente: se realizaron recorridos funcionales y revisión del autor, pero no un estudio con participantes ni acuerdo interevaluador.

La campaña es confiable dentro de su contrato porque cada cálculo puede reconstruirse desde runs.csv y campaign.json. No es infalible: un bug en el oráculo o el evaluador afectaría todas las puntuaciones. Por eso se conservan respuestas, referencias y metodología, y las ponderaciones se presentan de forma explícita. La posibilidad de auditar el método es más importante que una cifra aislada.

7.6. Selección de representantes y agregación

Cada combinación modelo-entrada contiene tres repeticiones. La agregación no puede elegir una corrida por su apariencia narrativa; utiliza criterios calculados y conserva la regla de selección. El representante debe ser una corrida válida con F1 primario disponible. Si ninguna repetición cumple, la celda permanece sin representante y su puntuación equilibrada es cero por ausencia de evidencia utilizable. Esta regla es severa, pero evita construir una recomendación a partir de una salida inválida o de un fallo que no dispone de métricas comparables.

La campaña 2 separa tres niveles. Primero se evalúa cada corrida: contrato, referencias, cobertura, claims, latencia y tokens. Después se agregan las repeticiones para obtener fiabilidad, medianas y estabilidad. Finalmente se calcula el índice por celda y se generan recomendaciones orientadas a una finalidad. Esta secuencia evita sumar observaciones heterogéneas sin denominador. Por ejemplo, la mediana de latencia se calcula sobre corridas válidas, mientras la fiabilidad conserva las tres intentadas; así, un modelo rápido que falla dos veces no aparece artificialmente como estable.

Los pesos del índice equilibrado son una convención explícita, no una verdad estadística. Fiabilidad recibe 35 puntos porque una combinación inutilizable con frecuencia no debe recomendarse aunque una salida aislada sea excelente. Evidencia recibe 25; seguridad

frente a claims no soportados, 20; eficiencia, 20. F1 y contrato pesan cero: F1 funciona como control del ground truth y el contrato como gate. El informe conserva las dimensiones separadas para que otra persona pueda discrepar de los pesos y reconstruir una decisión distinta sin repetir inferencias.

Tabla 10. Reglas de agregación de la campaña definitiva

Elemento	Regla	Motivo
Fiabilidad	Válidas / intentadas	Conservar todos los fallos
Latencia	Mediana de válidas	Reducir efecto de extremos
Tokens	Mediana de válidas	Comparar coste de salida
Contrato	Gate obligatorio	Una salida inválida no es recomendación
Representante	Corrida válida con F1 primario	Evitar selección narrativa
Celda sin válidas	Índice 0 y sin representante	No inventar evidencia
Recomendación	Por finalidad, no universal	Mantener dimensiones separadas

Fuente: metodología y resultados exportados por la campaña 2.

7.7. Amenazas a la validez experimental

La validez interna puede verse afectada por diferencias entre modelos, orden de ejecución, estado de caché y parámetros. El protocolo mitiga estos factores con warm-up por bloque, configuración congelada, captura del modelo observado y tres repeticiones. No los elimina por completo: Ollama y el sistema operativo administran memoria y planificación fuera del control de AURA. Por ello, las latencias describen el entorno capturado y no deben extrapolarse a otro equipo.

La validez de constructo depende de que las métricas representen el concepto que nombran. F1 mide cobertura del registro canónico, no capacidad de descubrir anomalías nuevas. Evidencia mide referencias visibles, no veracidad absoluta de cada explicación. Claims sin soporte detecta afirmaciones incompatibles con el payload según reglas programadas, pero puede omitir formulaciones indirectas. El índice equilibrado combina dimensiones para decisión; no es una medida científica única de 'inteligencia'. La memoria explicita estas fronteras para evitar que una cifra precisa oculte un constructo limitado.

La validez externa es la limitación principal. `synthetic_ground_truth.csv` contiene 15 filas y nueve columnas, construido para activar reglas conocidas. Los resultados permiten verificar el laboratorio y comparar comportamiento bajo ese contrato, pero no representan datasets grandes, multilingües, longitudinales o específicos de una industria. Una campaña con otro

dataset requiere nuevo ground truth, hashes y protocolo; reutilizar las puntuaciones actuales como garantía sería metodológicamente incorrecto.

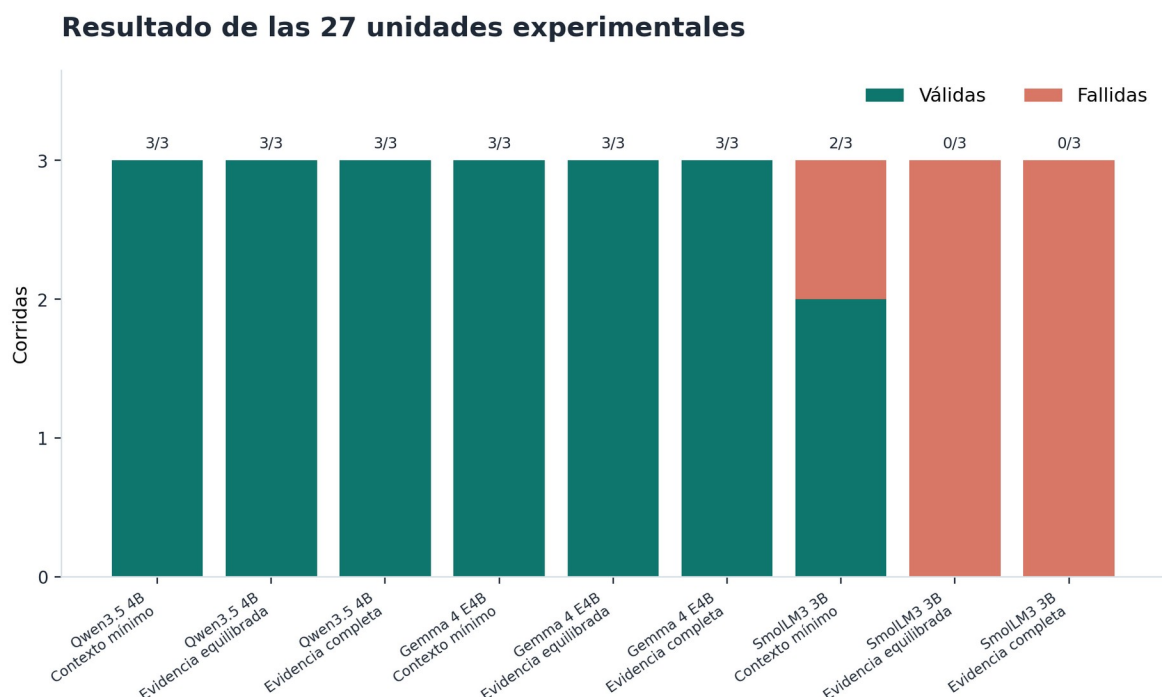
La validez de conclusión también es limitada por tres repeticiones. Las medianas y tasas describen la muestra, pero no sustentan intervalos estrechos ni pruebas de superioridad. La diferencia de 0,6 puntos entre dos celdas no debe interpretarse como dominancia general. En cambio, diferencias amplias de fiabilidad —como tres corridas válidas frente a ninguna— sí son operacionalmente relevantes para esta campaña. El análisis privilegia patrones consistentes y conserva valores exactos para evitar conclusiones basadas solo en el ranking.

8. Resultados y discusión

La campaña definitiva intentó las 27 unidades, completó 20 y conservó siete fallos. Las seis celdas de Qwen y Gemma alcanzaron 3/3 corridas válidas. SmolLM3 logró 2/3 con Contexto mínimo y 0/3 con Evidencia equilibrada y Evidencia completa. Todas las corridas válidas obtuvieron F1 1,0 contra el registro canónico. La diferencia entre combinaciones apareció en fiabilidad, contrato, soporte de evidencia, claims no soportados y eficiencia.

8.1. Panorama de la matriz

Figura 8. Corridas válidas y fallidas por combinación



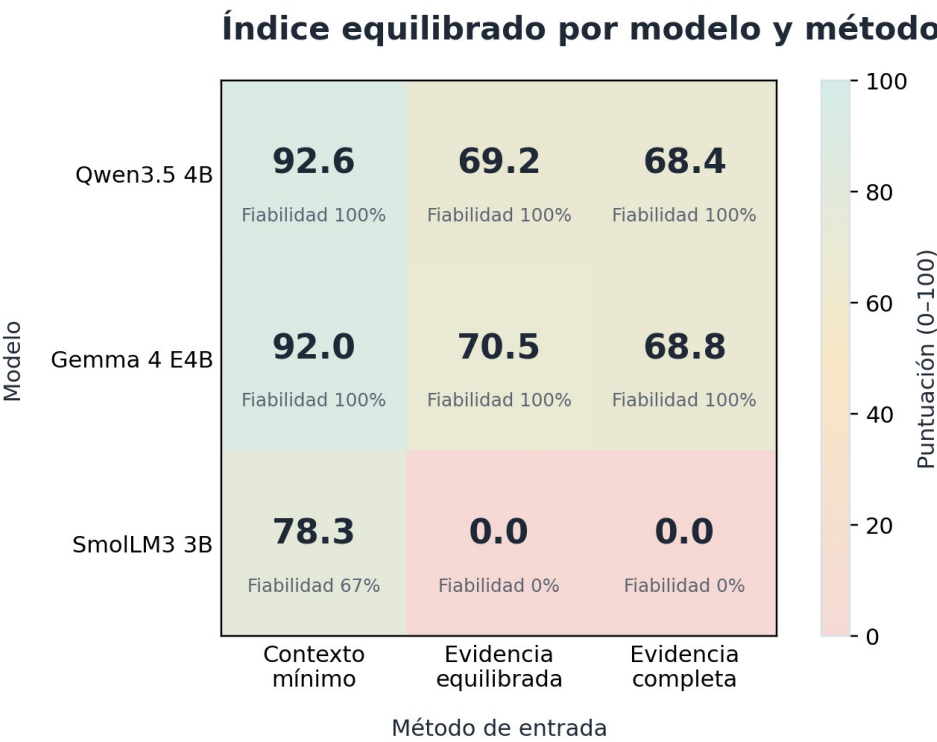
Fuente: elaboración propia a partir de results-summary.json, campaña 2.

El patrón más claro es la sensibilidad del modelo pequeño al aumento de contexto. SmolLM3 3B produjo dos diagnósticos válidos con Contexto mínimo, pero falló en las seis corridas de entradas más extensas. Qwen3.5 4B y Gemma 4 E4B fueron operativamente estables en las nueve corridas de cada modelo. La tasa global de éxito fue 20/27, equivalente a 74,1 %; ese valor no se presenta como propiedad general de Ollama, sino como resultado de esta matriz. Los siete fallos son parte del resultado. Eliminarlos habría favorecido artificialmente a SmolLM3 y ocultado que una opción rápida puede no soportar el contrato bajo entradas

largas. La fiabilidad actúa, por tanto, como dimensión separada de la calidad de las respuestas que sí lograron validarse.

8.2. Índice equilibrado

Figura 9. Índice equilibrado por modelo y método de entrada



Fuente: elaboración propia a partir de results-summary.json. El cero indica ausencia de corridas válidas, no medición de calidad narrativa.

Tabla 11. Resultados por modelo y método

Modelo	Método	Válidas	F1	Latencia mediana	Equilibrado
Qwen3.5 4B	Contexto mínimo	3/3	1,000	46,3 s	92,6
Qwen3.5 4B	Evidencia equilibrada	3/3	1,000	57,1 s	69,2
Qwen3.5 4B	Evidencia completa	3/3	1,000	57,9 s	68,4
Gemma 4 E4B	Contexto mínimo	3/3	1,000	48,9 s	92,0
Gemma 4 E4B	Evidencia equilibrada	3/3	1,000	50,7 s	70,5
Gemma 4 E4B	Evidencia completa	3/3	1,000	55,7 s	68,8
SmoLLM3 3B	Contexto mínimo	2/3	1,000	29,2 s	78,3
SmoLLM3 3B	Evidencia equilibrada	0/3	n/d	n/d	0,0
SmoLLM3 3B	Evidencia completa	0/3	n/d	n/d	0,0

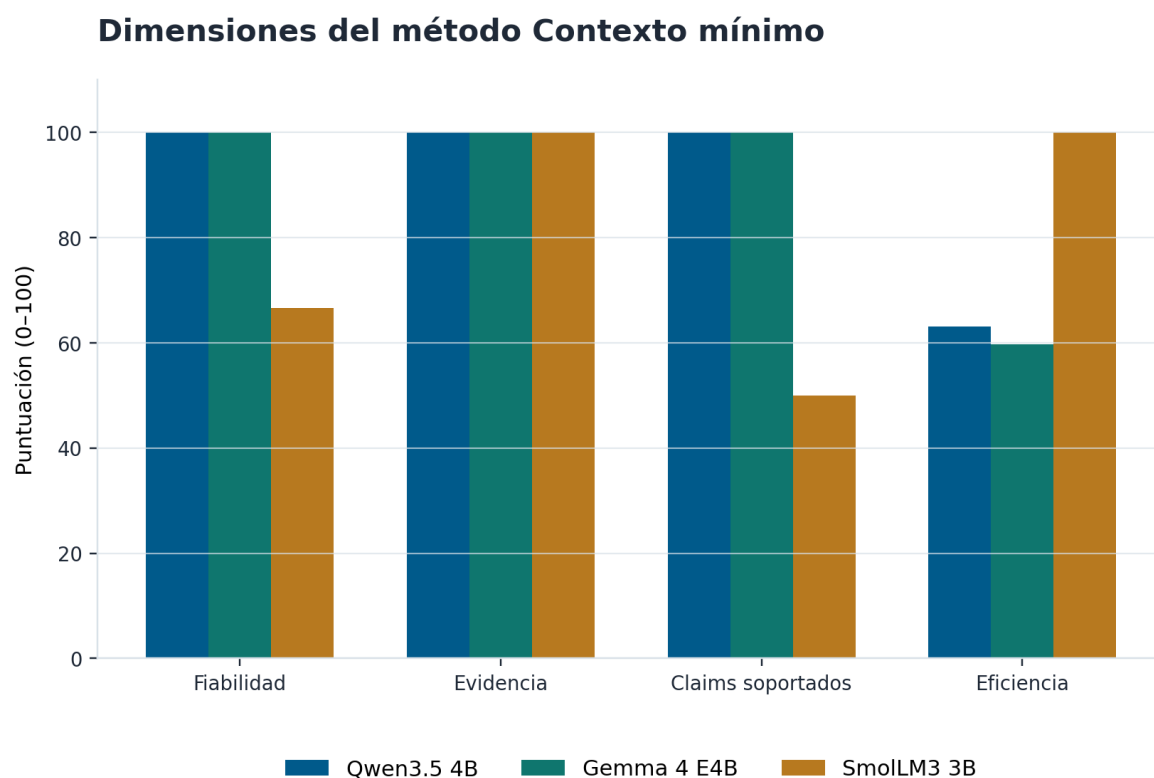
Fuente: report.md y results-summary.json, campaña 2.

Qwen3.5 4B con Contexto mínimo obtuvo el mayor índice equilibrado, 92,6, seguido muy de cerca por Gemma 4 E4B con el mismo método, 92,0. La diferencia de 0,6 puntos no justifica afirmar superioridad universal. Ambos completaron 3/3 corridas y alcanzaron 100 % en evidencia y seguridad frente a claims sin soporte; Qwen recibió mayor eficiencia relativa por su latencia mediana ligeramente menor.

Las entradas equilibrada y completa mantuvieron fiabilidad en Qwen y Gemma, pero registraron claims no soportados y, en algunas celdas, incumplimiento contractual. La penalización explica por qué más evidencia produjo menor índice. El hallazgo no significa que esas entradas sean inútiles; indica que, para este dataset explícito, aumentaron complejidad sin mejorar la cobertura ya exigida.

8.3. Calidad, evidencia y claims

Figura 10. Dimensiones del método Contexto mínimo



Fuente: elaboración propia a partir de results-summary.json.

Entre corridas válidas, la fidelidad de evidencia media fue 100 % y el anclaje medio 93,5 %. El anclaje no es idéntico en todos los modos porque el número de referencias visibles cambia. Contexto mínimo no expone bad samples; por tanto, su evaluación no puede exigir el mismo

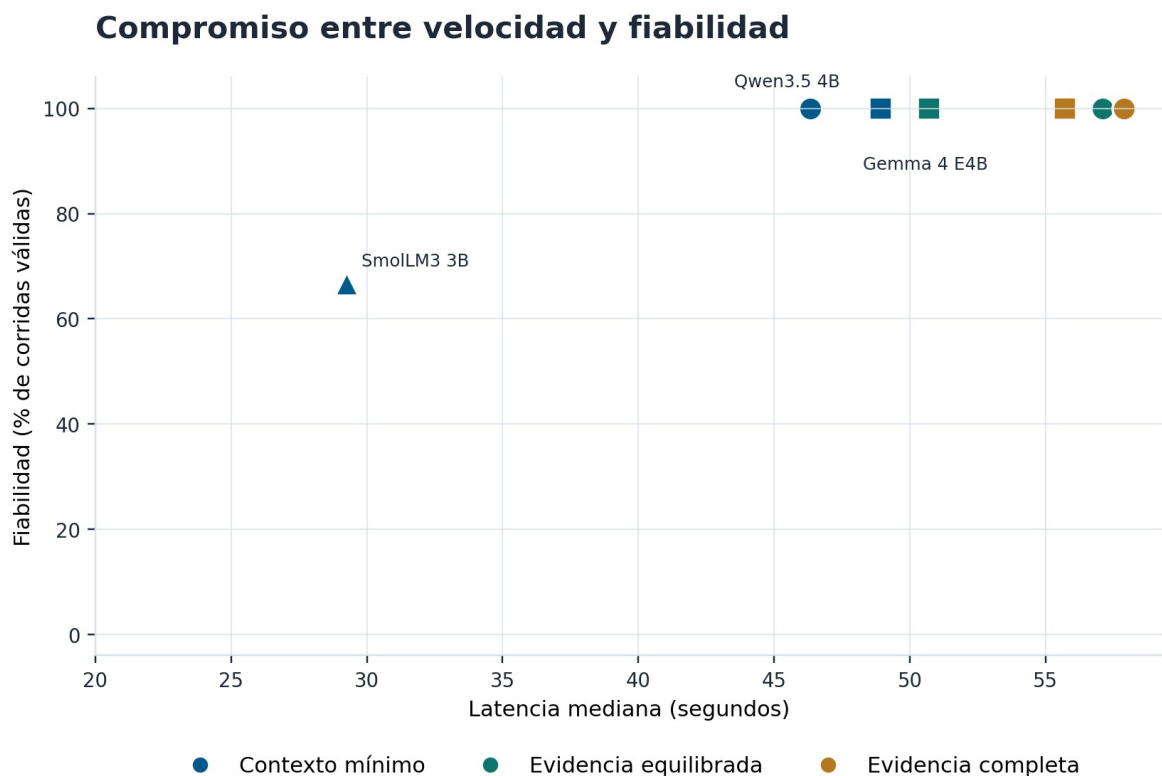
tipo de cita que Evidencia completa. El evaluador conoce el método efectivo y calcula solo lo observable.

Gemma 4 E4B con Contexto mínimo alcanzó 100 en la recomendación de calidad diagnóstica, fiabilidad y trazabilidad. Qwen3.5 4B fue seleccionado para equilibrio general. Esta aparente diferencia es intencional: la recomendación no fuerza un único ranking. Para un usuario que prioriza explicaciones soportadas y estabilidad, Gemma es una opción defendible; para quien busca equilibrio entre esas dimensiones y coste relativo, Qwen resultó mejor en esta campaña.

El F1 perfecto debe leerse con cautela. El modelo recibe un registro de incidencias que debe cubrir, por lo que la coincidencia con ground truth controla omisiones y referencias, pero no demuestra descubrimiento autónomo. La señal más discriminante estuvo en cómo cada modelo respetó el contrato y evitó agregar valores o explicaciones sin soporte. Esta lectura impide inflar el resultado del OE4.

8.4. Velocidad y estabilidad

Figura 11. Relación entre latencia mediana y fiabilidad



Fuente: elaboración propia a partir de results-summary.json.

La latencia mediana global fue 51,6 segundos y la salida mediana 3.559 tokens. SmolLM3 3B con Contexto mínimo fue la combinación más rápida, con 29,2 segundos, pero solo completó dos de tres corridas. Qwen y Gemma requirieron entre 46,3 y 57,9 segundos según entrada, con fiabilidad completa. La métrica de velocidad aislada habría recomendado SmolLM3; al incorporar estabilidad, su índice quedó en 78,3.

El resultado ilustra por qué no existe un ganador universal. Un flujo interactivo tolerante a reintentos puede preferir menor latencia; un proceso regulado puede priorizar que todas las corridas validen. AURA muestra ambas dimensiones y evita presentar la recomendación equilibrada como única respuesta.

8.5. Configuración recomendada y transferencia al pipeline

La configuración seleccionada para equilibrio general fue Qwen3.5 4B, Contexto mínimo, temperature 0,1, top_p 0,9, num_ctx 32.768, num_predict 8.192, think false, timeout 900 segundos y keep_alive 10 minutos. selected-configuration.json conserva estos valores y la interfaz permite copiarlos o aplicarlos al siguiente diagnóstico.

Figura 12. Configuración recomendada para el siguiente diagnóstico

DEL RESULTADO A LA PRÁCTICA

modelo disponible

Configuración para el próximo diagnóstico

La selección de la matriz y los gráficos se traduce aquí en parámetros exactos del pipeline normal.

MODELO OLLAMA

hf.co/unsloth/Qwen3.5-4B-GGUF:UD-Q4_K_XL

Contexto mínimo

TEMPERATURA	TOP_P	NUMCTX	NUMPREDICT
0.1	0.9	32.768	8.192
THINK	TIMEOUT	KEEP ALIVE	SEED
No	900s	10m	automática

ÍNDICE EQUILIBRADO	FIABILIDAD	EVIDENCIA	CORRIDAS VÁLIDAS
92.6/100	100.0/100	100.0/100	3/3

Configuración observada en esta campaña y este hardware. Aplicarla no garantiza el mismo resultado con otro dataset.

Copiar configuración

Aplicar al próximo diagnóstico

Ir a Auditoría

Fuente: AURA, campaña 2. La aplicación advierte que el resultado depende del dataset y del hardware.

La transferencia no convierte la campaña en una garantía. Si cambia el dataset, el modelo puede responder de forma distinta; si cambia el hardware, la latencia también. La práctica adecuada es utilizar la configuración como punto de partida y crear una nueva campaña cuando exista un nuevo dataset controlado con ground truth. El futuro orquestador de campañas deberá validar ese insumo antes de permitir comparaciones.

8.6. Resultado del pipeline de remediación

Además del Laboratorio, se validó un recorrido completo del pipeline normal con `synthetic_ground_truth.csv`. La persona aprobó cuatro acciones seguras —eliminar duplicados exactos y normalizar placeholders en edad, email y estado— y rechazó once acciones que requerían decisión de dominio o privacidad. El script determinista se ejecutó sobre una copia, redujo las filas de 15 a 14 y conservó nueve columnas.

La reauditoría pasó de 15 a 11 hallazgos. Se corrigieron duplicados exactos y activaciones de placeholders; persistieron nulos, mojibake, outliers, negativos, email inválido, fechas mixtas y PII. No aparecieron reglas nuevas. Este resultado es coherente con las cuatro acciones aprobadas: AURA no promete corregir hallazgos rechazados. La evidencia demuestra trazabilidad de la ejecución, no calidad semántica universal.

El score permaneció en cero porque el algoritmo de puntuación penaliza la presencia de varios hallazgos críticos y no es lineal con el número de reglas corregidas. Por ello, el delta de hallazgos y la lista de reglas son más informativos que el score agregado en este caso. La memoria evita presentar 15→11 como una mejora de 26,7 % en calidad total; es una reducción de activaciones deterministas dentro de una copia controlada.

8.7. Respuesta a las preguntas de investigación

Tabla 12. Respuesta sintética a las preguntas de investigación

Pregunta	Resultado	Evidencia	Conclusión acotada
RQ1	Motor y ground truth reproducibles	Activaciones y métricas	Detecta el alcance anotado; no todo dominio
RQ2	20/27 diagnósticos válidos	Respuestas, recibos y scoring	Qwen/Gemma estables; SmolLM sensible al contexto
RQ3	Ejecución y reauditoría verificadas	Script, bundle, receipt, corrected.csv	Corrige acciones aprobadas sobre una copia
RQ4	Recomendaciones por finalidad	Índice y dimensiones	No existe ganador universal

Elaboración propia.

La respuesta general es afirmativa dentro del alcance: es posible combinar auditoría determinista, diagnóstico LLM y remediación verificable sin ceder autoridad al modelo. La arquitectura no elimina los riesgos del LLM; los contiene mediante contexto controlado, validación, recibos, catálogo cerrado de acciones y supervisión humana. La campaña muestra que la selección de modelo debe basarse en una finalidad y en datos controlados, no en popularidad o tamaño.

8.8. Lectura de resultados por pregunta de investigación

Respecto a RQ1, el dataset controlado cumple su función como oráculo: las corridas válidas alcanzan cobertura exacta del registro canónico. Este resultado confirma que el pipeline puede comparar salidas contra una referencia, no que el motor cubra cualquier anomalía imaginable. La interpretación correcta es instrumental: el ground truth permite saber si una respuesta omitió o inventó elementos bajo el contrato definido.

Para RQ2, la evidencia es mixta y por ello útil. Qwen3.5 4B y Gemma 4 E4B completaron sus nueve corridas, mientras SmoLLM3 3B solo produjo dos válidas en Contexto mínimo y ninguna en los otros métodos. Las salidas válidas mantuvieron alta fidelidad de evidencia y anclaje. Los siete fallos muestran que un modelo más pequeño puede ser veloz, pero no necesariamente sostener un JSON largo y gobernado. La conclusión no depende de una impresión textual: está respaldada por fiabilidad, errores conservados y recibos de modelo observado.

Para RQ3, el flujo de producto demostró que una selección humana de cuatro acciones puede convertirse en script determinista, ejecutarse sobre una copia y reauditar. El resultado pasó de 15 a 11 activaciones: duplicados y placeholders se redujeron, mientras reglas de dominio y privacidad persistieron. La reducción es coherente con las acciones aprobadas y no se presenta como limpieza total. La evidencia del runner, los hashes y el delta antes/después responden a la pregunta de trazabilidad, aunque no sustituyen una evaluación semántica externa.

Para RQ4, el índice equilibrado recomienda Qwen3.5 4B con Contexto mínimo, pero las recomendaciones por finalidad aportan una lectura más completa. Gemma 4 E4B con Contexto mínimo resulta apropiado cuando se priorizan calidad y trazabilidad; SmoLLM3 3B solo aparece para velocidad, acompañado de una advertencia de fiabilidad. La capacidad de copiar o aplicar una configuración convierte la matriz en una decisión operativa. AURA debe mostrar siempre dataset y entorno junto a esa recomendación para que el usuario no confunda conveniencia local con superioridad universal.

Tabla 13. Síntesis de respuestas a las preguntas de investigación

Pregunta	Evidencia principal	Respuesta delimitada
RQ1 Motor	GT y cobertura exacta	Reproduce el registro anotado
RQ2 Diagnóstico	20 válidas, 7 fallos, evidencia y anclaje	Qwen y Gemma son estables en este protocolo

Pregunta	Evidencia principal	Respuesta delimitada
RQ3 Remediación	Script, receipt y 15→11	La cadena es verificable sobre una copia
RQ4 Selección	Índice y recomendaciones por finalidad	No existe ganador universal

Elaboración propia.

8.9. Implicaciones prácticas y escenario de uso

Un escenario razonable comienza con un analista que recibe un CSV cuya calidad desconoce. Primero ejecuta el perfil base sin modelo y obtiene hechos reproducibles. Si necesita interpretación, consulta la recomendación vigente del Laboratorio o ejecuta una campaña con un dataset controlado comparable. Selecciona modelo y método, revisa el diagnóstico y exporta el informe. Si decide corregir, entra en la rama opcional, aprueba únicamente acciones respaldadas por su conocimiento de dominio, revisa el script y lo ejecuta sobre una copia. La reauditoría le muestra qué reglas cambiaron antes de publicar o entregar el resultado.

En este escenario, el valor de AURA no reside solo en detectar un dato anómalo. Reside en coordinar momentos de autoridad: el motor afirma lo que puede medir; el modelo explica lo visible; la persona decide; el renderer traduce una acción cerrada; el runner prueba qué se ejecutó; y el motor vuelve a medir. La exportación conserva cada transición. Esta estructura permite discutir un resultado con otra persona sin depender de la memoria del operador ni de una captura aislada del chat con el modelo.

La campaña también tiene un uso limitado pero concreto. Antes de integrar un modelo en el pipeline, el equipo puede construir un ground truth representativo, definir parámetros y observar fiabilidad, evidencia y coste. El resultado no reemplaza pruebas sobre el dataset real, pero reduce decisiones basadas solo en popularidad o tamaño del modelo. El orquestador futuro deberá exigir el insumo básico —dataset, anotación y esquema— y advertir cuando la campaña sea meramente exploratoria por carecer de referencia.

Desde la perspectiva de adopción, la interfaz necesita dos niveles de lectura. El primero debe responder qué ocurrió, qué combinación conviene y por qué. El segundo debe permitir abrir hashes, prompts, recibos y errores. La campaña 1 mostró que una pantalla técnicamente completa puede ser poco clara si el progreso, los estados y los fallos no tienen jerarquía visual. La campaña 2 incorporó matriz con estados, respuesta en streaming, visualizador de

resultados, glosario y configuración transferible. Esta evolución forma parte de la evaluación del producto, no solo de su estética.

8.10. Contraste con el estado del arte

Frente a las herramientas centradas en reglas, AURA añade interpretación contextual y una rama de remediación gobernada, pero conserva las reglas como fuente de hechos. Frente a asistentes generativos de datos, restringe referencias y código para reducir claims no soportados. Frente a plataformas de observabilidad, ofrece menor alcance organizacional, aunque una cadena local más fácil de inspeccionar en un prototipo académico. Frente a benchmarks generales de LLM, su Laboratorio es pequeño y específico, pero mide exactamente el contrato que consume el producto.

La propuesta coincide con la literatura multidimensional de calidad: una puntuación agregada no basta para decidir. También coincide con HELM en exponer escenarios, dimensiones y ausencias. Se diferencia al incorporar la recomendación al flujo operativo: un resultado puede copiarse como modelo, método y parámetros para el siguiente diagnóstico. Esa transferencia es valiosa siempre que conserve la etiqueta contextual. El sistema no debería ocultar que la recomendación proviene de `synthetic_ground_truth.csv` y de un entorno Windows concreto.

El enfoque local-first aporta una propiedad que las métricas no capturan por completo: control sobre el archivo y capacidad de operar sin enviar el CSV a un proveedor remoto. No obstante, local no equivale automáticamente a seguro. El equipo debe configurar orígenes de Ollama, revisar exportaciones y proteger el host. AURA trata local-first como una decisión de arquitectura y minimización, no como una certificación. Esta lectura evita convertir una ventaja de despliegue en una afirmación absoluta de privacidad.

9. Conclusiones

AURA se consolidó como un prototipo funcional de auditoría inteligente de calidad del dato con arquitectura local-first. El sistema integra perfilamiento determinista, diagnóstico estructurado, evidencia exportable, remediación gobernada, script reproducible, ejecución sobre copia y reauditoría. El Laboratorio reutiliza el mismo diagnóstico para comparar modelos y entradas sin introducir script ni HITL en la puntuación.

9.1. Cumplimiento del objetivo general

El objetivo general se considera alcanzado en el alcance del TFM: AURA puede cargar y auditar un CSV localmente, producir evidencia, restringir la intervención del modelo, mantener decisiones humanas y demostrar una cadena de integridad hasta el CSV corregido. La aplicación y sus exportaciones permiten reconstruir qué se observó, qué se recomendó, qué se aprobó, qué código se ejecutó y qué cambió después.

El calificativo ‘inteligente’ no se asocia a autonomía. Describe la capacidad de combinar reglas, estadística y explicación contextual, manteniendo al modelo dentro de una frontera. Esta interpretación es más sólida que una limpieza generativa libre y responde al problema de responsabilidad sobre transformaciones.

9.2. Cumplimiento de los objetivos específicos

Tabla 14. Trazabilidad de los seis objetivos específicos

Objetivo	Componente	Evidencia	Estado
OE1 Local-first	Carga, perfil y Ollama local	Capturas, snapshot y exportación	Cumplido
OE2 Motor determinista	Reglas, tipos y métricas	Ground truth, tests y reauditoría	Cumplido con alcance anotado
OE3 Diagnóstico restringido	Envelope, prompt y JSON V2	Recibo, hashes y validator	Cumplido
OE4 Laboratorio	Matriz 3×3×3 y scoring	Campañas 1 y 2	Cumplido en un dataset
OE5 HITL	Plan, aprobación y política	Contrato aprobado y E2E	Cumplido en pipeline normal
OE6 Script trazable	Renderer, runner y reauditoría	Script, bundle, receipt, corrected.csv	Cumplido en copia controlada

Elaboración propia.

OE4 no se sostiene solo porque exista una pantalla de Laboratorio. Se sostiene porque la campaña definitiva conserva 27 corridas, fallos, modelos observados, parámetros, artefactos y método de puntuación. OE5 y OE6 tampoco se confunden con el Laboratorio: su evidencia procede del pipeline de remediación y del recorrido E2E real.

9.3. Aportaciones del trabajo

La primera aportación es una frontera de autoridad implementada. El modelo interpreta; AURA valida y propone acciones cerradas; la persona decide; el renderer genera; el runner ejecuta; y el motor reaudita. La segunda es un expediente reproducible que vincula artefactos mediante hashes y manifiestos. La tercera es un Laboratorio contextual que recomienda configuraciones por finalidad y permite trasladarlas al uso normal.

La cuarta aportación es metodológica: la memoria documenta fallos y correcciones del piloto. En lugar de presentar una línea de éxito continua, muestra cómo una campaña descubrió discrepancias entre runner y producto, problemas de interfaz y sesgos del índice. La campaña 2 es más defendible precisamente porque esas observaciones se incorporaron antes de congelar el protocolo.

9.4. Síntesis final

El resultado central no es que un modelo concreto ‘gane’, sino que AURA convierte una decisión difusa en una comparación auditable. Para `synthetic_ground_truth.csv` y el entorno capturado, Qwen3.5 4B con Contexto mínimo ofreció el mejor equilibrio; Gemma 4 E4B fue una alternativa igualmente estable con mejores recomendaciones de calidad y trazabilidad; y SmolLM3 3B mostró ventaja de velocidad con menor robustez. Otra tarea, dataset o equipo puede cambiar esa conclusión.

AURA demuestra que la incorporación de LLM a la calidad del dato puede ser útil sin convertir al modelo en autoridad. La reproducibilidad no proviene de que la respuesta sea idéntica, sino de conservar la entrada, validar el contrato, registrar el modelo observado, contabilizar fallos y limitar las consecuencias de una salida probabilística.

10. Limitaciones y trabajo futuro

10.1. Limitaciones

- La campaña utiliza un único dataset controlado de 15 filas y nueve columnas; no sustenta generalización a datasets reales grandes o a otros dominios.
- Tres repeticiones por celda permiten descripción y comparación inicial, pero no inferencia causal ni estimación precisa de varianza.
- Todos los modelos se ejecutan cuantizados mediante Ollama en un único entorno Windows; latencia y estabilidad dependen de hardware, runtime y versión.
- El ground truth controla cobertura de un registro conocido. $F1=1,0$ no demuestra descubrimiento independiente ni ausencia total de defectos.
- El evaluador es determinista pero puede contener errores de implementación; su confiabilidad depende de tests, revisión y artefactos abiertos.
- La evaluación de usabilidad no incluye participantes externos ni un instrumento validado; solo existen recorridos funcionales y revisión del autor.
- La remediación validada cubre una copia y cuatro acciones aprobadas. No se afirma que cualquier script o decisión humana mejore semánticamente un dataset.
- El uso de asistentes de IA durante el desarrollo y la redacción se declara, pero no contó con autorización previa formal del director; requiere revisión antes del depósito.

10.2. Trabajo futuro

La prioridad es crear un orquestador de campañas para nuevos datasets. El usuario deberá proporcionar CSV, esquema y ground truth con validaciones de cobertura. AURA generará el protocolo, congelará hashes y permitirá elegir modelos y parámetros antes de iniciar. Sin ground truth, la aplicación podrá ejecutar un piloto exploratorio, pero no presentarlo como campaña comparativa formal.

La segunda línea es ampliar validación externa: datasets anonimizados de distintos dominios, más reglas negativas, más repeticiones, cuantizaciones alternativas y hardware diverso. También conviene incorporar intervalos de confianza y análisis de sensibilidad de las ponderaciones del índice equilibrado.

La tercera línea es una evaluación de usabilidad con participantes. Las tareas deberían cubrir carga, comprensión de hallazgos, elección de método, aprobación de acciones, ejecución externa y lectura de la reauditoría. Se pueden medir tiempo, errores, éxito por tarea, confianza y comprensión de límites. Un estudio de este tipo permitiría evaluar si la trazabilidad técnica resulta comprensible fuera del equipo de desarrollo.

La cuarta línea es robustecer seguridad: pruebas adversariales de prompt injection en nombres y valores, límites de persistencia, aislamiento del runner, firma de expedientes y análisis de dependencias. La arquitectura ya trata el payload como contenido no confiable, pero requiere evaluación sistemática frente a ataques y datasets maliciosos.

10.3. Riesgos de adopción

El principal riesgo es interpretar la recomendación del Laboratorio como certificación universal. La interfaz debe mantener visible el dataset, entorno, modelo, entrada y parámetros. Otro riesgo es que la persona apruebe una remediación sin conocimiento de dominio; la revisión humana es una frontera de autoridad, no garantía de decisión correcta. Finalmente, la facilidad para exportar evidencia puede inducir a compartir muestras sensibles; el expediente requiere revisión antes de salir del equipo.

Estos riesgos no invalidan la propuesta, pero delimitan su uso. AURA es una herramienta de apoyo y defensa de decisiones de calidad del dato, no un sistema autónomo de gobierno del dato ni un sustituto de políticas organizacionales.

Referencias bibliográficas

- Autio, C., Schwartz, R., Dunietz, J., Jain, S., Stanley, M., Tabassi, E., Hall, P., & Roberts, K. (2024). Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile (NIST AI 600-1). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.AI.600-1>
- Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency, 610–623. <https://doi.org/10.1145/3442188.3445922>
- Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). CRISP-DM 1.0: Step-by-step data mining guide. SPSS.
- European Parliament and Council. (2016). Regulation (EU) 2016/679 (General Data Protection Regulation). Official Journal of the European Union.
- European Parliament and Council. (2024). Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence. Official Journal of the European Union.
- Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2022). GPTQ: Accurate post-training quantization for generative pre-trained transformers. arXiv. <https://doi.org/10.48550/arXiv.2210.17323>
- ISO/IEC. (2008). ISO/IEC 25012:2008 Software engineering—Software product Quality Requirements and Evaluation (SQuARE)—Data quality model. International Organization for Standardization.
- JSON Schema. (2022). JSON Schema Draft 2020-12. <https://json-schema.org/draft/2020-12>
- Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). Local-first software: You own your data, in spite of the cloud. Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software, 154–178. <https://doi.org/10.1145/3359591.3359737>
- Liang, P., Bommasani, R., Lee, T., et al. (2022). Holistic evaluation of language models. arXiv. <https://doi.org/10.48550/arXiv.2211.09110>
- Lin, J., Tang, J., Tang, H., et al. (2024). AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. Proceedings of Machine Learning and Systems, 6.
- National Institute of Standards and Technology. (2015). Secure Hash Standard (SHS) (FIPS PUB 180-4). <https://doi.org/10.6028/NIST.FIPS.180-4>
- Ollama. (2024, December 6). Structured outputs. <https://ollama.com/blog/structured-outputs>
- Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. 2022 IEEE Symposium on Security and Privacy, 754–768. <https://doi.org/10.1109/SP46214.2022.9833571>

- Pipino, L. L., Lee, Y. W., & Wang, R. Y. (2002). Data quality assessment. *Communications of the ACM*, 45(4), 211–218. <https://doi.org/10.1145/505248.506010>
- Rundgren, A., Jordan, B., & Erdtman, S. (2020). JSON Canonicalization Scheme (JCS) (RFC 8785). RFC Editor. <https://doi.org/10.17487/RFC8785>
- Schwaber, K., & Sutherland, J. (2020). The Scrum Guide. <https://scrumguides.org>
- Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4), 427–437. <https://doi.org/10.1016/j.ipm.2009.03.002>
- Tabassi, E. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0) (NIST AI 100-1). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.AI.100-1>
- Wang, R. Y., & Strong, D. M. (1996). Beyond accuracy: What data quality means to data consumers. *Journal of Management Information Systems*, 12(4), 5–33. <https://doi.org/10.1080/07421222.1996.11518099>

Anexo A. Parámetros y configuración reproducible

Tabla 15. Configuración seleccionada para el pipeline normal

Parámetro	Valor	Función
Modelo	Qwen3.5 4B UD-Q4_K_XL	Combinación equilibrada
Método	Contexto mínimo	Menor payload
temperature	0,1	Reduce variación
top_p	0,9	Limita masa de probabilidad
num_ctx	32.768	Ventana de contexto
num_predict	8.192	Máximo de salida
think	false	Evita razonamiento adicional
timeout	900 s	Tiempo máximo
keep_alive	10 m	Mantiene modelo cargado
seed	automática	No fija repetición exacta

Fuente: *selected-configuration.json*, campaña 2.

La configuración es evidencia de una campaña, no recomendación universal. Debe probarse nuevamente cuando cambien dataset, modelo, runtime o hardware.

Anexo B. Glosario para lectura de resultados

Tabla 16. Glosario operativo

Término	Explicación
Ground truth (GT)	Referencia conocida contra la que se compara la salida.
TP	Hallazgo esperado que el sistema declaró correctamente.
FP	Hallazgo declarado que la referencia considera incorrecto.
FN	Hallazgo esperado que no fue declarado.
Precisión	De lo declarado por el sistema, qué proporción fue correcta.
Recall	De lo que se esperaba, qué proporción apareció.
F1	Media armónica de precisión y recall; penaliza desequilibrio.
Fiabilidad	Proporción de corridas válidas entre intentadas.
Contrato	Reglas estructurales y de referencia que debe cumplir el JSON.
Evidencia	Información visible que respalda una afirmación.
Anclaje	Uso correcto de referencias del envelope.
Claim sin soporte	Afirmación que no puede verificarse con la entrada visible.
Alucinación	Columna, valor o relación inventada o no respaldada.
Latencia	Tiempo total de una corrida.
Tokens	Unidades de texto procesadas o generadas.
Contexto mínimo	Resumen, esquema y registro mínimo; sin muestras.
Evidencia equilibrada	Añade estadísticas, reglas y muestras limitadas.
Evidencia completa	Añade políticas, manifiestos y anclajes exactos.
num_ctx	Tamaño máximo de la ventana de contexto.
num_predict	Máximo de tokens que puede generar el modelo.
Índice equilibrado	Combinación ponderada para ayuda a la decisión.

Adaptado de *glossary.md* de la campaña 2 y de Sokolova y Lapalme (2009).

Anexo C. Artefactos y rutas de evidencia

Tabla 17. Rutas de evidencia utilizadas en la memoria

Evidencia	Ruta relativa	Uso
Campaña piloto	experiments/tests/test_campaña/	Calibración e incidencias
Campaña definitiva	experiments/tests/campana2/ resultado_export/	Resultados formales
Capturas campaña 2	experiments/tests/campana2/*.png	Interfaz y visualización
Ejecución verificada	docs/tercera_entrega_aura/ 03_evidencia/screenshots/apply-verify/	Pipeline normal
Memoria final	docs/tfm/memoria_final/	Documento y requisitos

Elaboración propia.

Los nombres internos de los métodos permanecen en archivos exportados por compatibilidad: prompt_libre = Contexto mínimo; smart_sample = Evidencia equilibrada; recommended = Evidencia completa.

Anexo D. Frontera de claims

Tabla 18. Afirmaciones permitidas y no permitidas

Evidencia	Afirmación permitida	Afirmación no permitida
F1=1,0	Cubrió el registro canónico	Descubrió todos los defectos
20/27 válidas	Fiabilidad observada de 74,1 %	Fiabilidad universal
92,6 equilibrado	Mejor combinación en esta campaña	Mejor modelo en general
15→11 hallazgos	Se redujeron cuatro activaciones	El dataset quedó limpio
Hash válido	El artefacto no cambió	El contenido es correcto
Revisión humana	Una persona autorizó la acción	La decisión es semánticamente perfecta

Elaboración propia.

Anexo E. Declaración ampliada de herramientas

Durante el desarrollo se emplearon asistentes de IA para proponer organización de tareas, revisar cambios de código, generar casos de prueba, sintetizar resultados y apoyar la redacción. El autor decidió qué cambios aceptar, ejecutó o supervisó las campañas, conservó los artefactos y realizó la interpretación final. Las respuestas de los asistentes no se utilizaron como fuente bibliográfica ni como medición del producto.

Para esta versión, las afirmaciones académicas se contrastaron con normas, legislación, documentación oficial y artículos citados. Las cifras se obtuvieron de artefactos locales. La declaración se incluye porque la guía institucional exige transparencia sobre el uso de herramientas de IA y porque el autor no gestionó previamente una autorización formal con el director. Antes del depósito debe solicitarse la revisión expresa del director.